

Citation for published version:
Crick, T 2004, *A GCC front end for BCPL*. Computer Science Technical Reports, no. CSBU-2004-06, Department of Computer Science, University of Bath.

Publication date:
2004

Link to publication

©The Author May 2004

University of Bath

Alternative formats
If you require this document in an alternative format, please contact:
openaccess@bath.ac.uk

General rights
Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

Take down policy
If you believe that this document breaches copyright please contact us providing details, and we will remove access to the work immediately and investigate your claim.

Download date: 24. Oct. 2026

Department of Computer Science



Technical Report

Undergraduate Dissertation: A GCC Front End for BCPL

Tom Crick

Technical Report CSBU-2004-06 **May 2004**

ISSN 1740-9497

Copyright ©May 2004 by the authors.

Contact Address:
Department of Computer Science
University of Bath
Bath, BA2 7AY
United Kingdom
URL: http://www.cs.bath.ac.uk

ISSN 1740-9497

A GCC Front End for BCPL

Tom Crick
tc@cs.bath.ac.uk
BSc (Hons) Computer Science
University of Bath

May 2004

Abstract

The BCPL programming language was developed by Martin Richards at the University of Cambridge in 1967. It was a simplified version of the earlier language CPL and over the following fifteen years became a popular language for system programming. It was typeless, block structured and gave the programmer enormous power and flexibility. The development of its descendant C as the language of choice for system programming led to its decline, but large amounts of legacy code exists today. This dissertation attempts to apply modern compilation techniques to craft a BCPL compiler and also investigates a number of recent developments in the GNU Compiler Collection (GCC). A solution is proposed by developing a GCC front end for BCPL using the optimisation framework for trees based on the Static Single Assignment (SSA) form. It is concluded that the implementation would provide legacy BCPL users with access to a popular, robust and multi-platform compiler infrastructure.

A GCC Front End for BCPL

submitted by Tom Crick

COPYRIGHT

Attention is drawn to the fact that the copyright of this dissertation rests with its author. The Intellectual Property Rights of the products produced as part of the project belong to the University of Bath (see `http://www.bath.ac.uk/ordinances/#intelprop`).

This copy of the dissertation has been supplied on condition that anyone who consults it is understood to recognise that its copyright rests with its author and that no quotation from the dissertation and no information derived from it may be published without the prior written consent of the author.

DECLARATION

This dissertation is submitted to the University of Bath in accordance with the requirements of the degree of Bachelor of Science in the Department of Computer Science. No portion of this work in this dissertation has been submitted in support of an application for any other degree or qualification of this or any other university of institution of learning. Except where specifically acknowledged, it is the work of the author.

This thesis may be made available for consultation within the University Library and may be photocopied or lent to other libraries for the purpose of consultation.

Signed: .....

Acknowledgements

I would like to say a massive thank you to Professor John Fitch, whose experience and guidance during the project was invaluable. I would also like to thank my girlfriend Kate and my parents for their support over the past four years.

Contents

1 Introduction **1**
- 1.1 Aim . . . 1
- 1.2 Objectives . . . 1
- 1.3 Structure . . . 1

2 Literature Survey **2**
- 2.1 History . . . 2
  - 2.1.1 CPL . . . 2
  - 2.1.2 BCPL . . . 2
  - 2.1.3 Development of B and C . . . 3
  - 2.1.4 Code Examples . . . 4
- 2.2 Compiler Phases . . . 4
- 2.3 Static Single Assignment Form . . . 5
- 2.4 GNU Compiler Collection Overview . . . 6
  - 2.4.1 GCC Front Ends . . . 6
  - 2.4.2 GCC Intermediate Representations . . . 7
- 2.5 Possible Implementations . . . 8
  - 2.5.1 Flex and Bison . . . 8
  - 2.5.2 BCPL to C Conversion . . . 8
  - 2.5.3 Norcroft C Compiler . . . 9
  - 2.5.4 lcc, A Re-targetable Compiler for ANSI C . . . 9
  - 2.5.5 The Stanford University Intermediate Format Project . . . 9
  - 2.5.6 The Low Level Virtual Machine Project . . . 10
  - 2.5.7 GNU Compiler Collection . . . 11
  - 2.5.8 Existing Work . . . 11
- 2.6 Design Issues . . . 13
  - 2.6.1 BCPL Syntax Features . . . 13
  - 2.6.2 Back End Phases . . . 13
  - 2.6.3 Coding Standards . . . 13
  - 2.6.4 Legal . . . 14

3 Requirements **15**
- 3.1 Requirements Analysis . . . 15
- 3.2 Requirements Specification . . . 15
  - 3.2.1 System Perspective . . . 15
  - 3.2.2 System Features . . . 15
  - 3.2.3 User Characteristics . . . 17
  - 3.2.4 Operating Environment . . . 17
  - 3.2.5 Documentation . . . 18

4 Design **19**
- 4.1 System Architecture . . . 19
- 4.2 Methodology . . . 19
- 4.3 Lexical Analyser . . . 19
- 4.4 Symbol Table . . . 21
- 4.5 Parser . . . 22
- 4.6 Intermediate Representations . . . 24
- 4.7 GCC Integration . . . 24

5 Implementation **25**
5.1 Overview . . . 25
5.2 Tools . . . 25
5.3 Lexical Analyser . . . 25
5.4 Symbol Table . . . 27
5.5 Parser . . . 28
5.6 Intermediate Representations . . . 31
5.6.1 Overview . . . 31
5.6.2 Trees . . . 32
5.6.3 Identifiers . . . 32
5.6.4 Declarations . . . 33
5.6.5 Functions . . . 33
5.6.6 Statements . . . 34
5.6.7 Expressions . . . 34
5.6.8 Global Vector . . . 35
5.6.9 System Library . . . 35
5.6.10 Miscellaneous . . . 36
5.7 GCC Integration . . . 37

6 System Testing **39**

7 Conclusion **41**
7.1 Overview . . . 41
7.2 Milestones . . . 42
7.3 Further Work . . . 43
7.4 Concluding Remarks . . . 43

Bibliography **44**

Appendices **47**

A Compiler Phases **48**

B BCPL, B and C Code Examples **49**

C Backus-Naur Form Grammar for BCPL **50**

D BCPL Operator Precedence **53**

E Integration of GENERIC and GIMPLE into GCC **54**

F Test Results **57**

G Program Versions **60**

H Source Code **61**

List of Figures

2.1 Simple SSA form example . . . 5
2.2 Example RTL S-expression . . . 7
2.3 GCC implementation of SSA . . . 8

4.1 Compiler system overview . . . 20
4.2 Separately-chained hash table composition . . . 22
4.3 Nested code blocks example . . . 23

5.1 Example set of Flex rules for BCPL . . . 26
5.2 Example set of Flex regular expressions for BCPL . . . 26
5.3 Symbol table data structures . . . 28
5.4 Symbol table hash function . . . 28
5.5 Example Bison production rules for BCPL . . . 29
5.6 Example of left-recursion in Bison production rule for BCPL . . . 29
5.7 Syntactic ambiguities in BCPL repetitive commands . . . 31

A.1 Basic compiler phases . . . 48

B.1 BCPL code example . . . 49
B.2 B code example . . . 49
B.3 C code example . . . 49

D.1 BCPL operator precedence and associativity . . . 53

E.1 Existing GCC framework . . . 55
E.2 Integration of GENERIC and GIMPLE into GCC . . . 56

Chapter 1

Introduction

1.1 Aim

The aim of this dissertation is to develop a modern compiler infrastructure for the BCPL system programming language.

1.2 Objectives

- To provide a compiler for the BCPL language as defined by the creator Martin Richards (Richards 1967).
- To provide for some of the commonly-used extensions of the language.
- To investigate popular compiler infrastructures and analyse how they could be used to provide a framework for a BCPL compiler.
- To develop a modern compilation strategy for BCPL legacy users by developing a BCPL front end for GCC.

1.3 Structure

Chapter 2 discusses in-depth the history and heritage of BCPL and details some of the more important language features. It also investigates previous work in developing a BCPL compiler and discusses a wide range of approaches to the problem. In Chapter 3 the process of requirements elicitation and capture is examined with respect to the problem at hand, followed by an overview of the software and design decisions in Chapter 4. Chapter 5 discusses in detail the chosen implementation of the project and reflects on how specific difficulties were overcome. The process of testing the system, followed by concluding remarks and future work is in Chapters 6 and 7 respectively.

Chapter 2

Literature Survey

2.1 History

2.1.1 CPL

The publication in 1963 of the revised ALGOL report (Naur, Backus & McCarthy 1963) marked an enormous step forward in the design of programming languages. Probably the most important idea, which arose almost by accident, was that of block structure and the stack mechanism for implementing it. A block is a set of data definitions followed by a sequence of commands and may be nested one within another. Each data definition implies a scope, over which the defined term is visible, i.e. where it may be legally referred to in either commands or other definitions. Outside this scope, the item has no existence. Thus memory space need be allocated for data only while there are commands to process it; and data names can be reused in different contexts (Fischer & LeBlanc 1991).

During the decade that followed the ALGOL report, several new languages were devised, each claiming some superiority over ALGOL 60, but all borrowing ideas from it, including that of block structure. CPL (Combined Programming Language) was a collaborative venture between the University of London Institute of Computer Science and the Cambridge University Mathematical Laboratory. CPL was never fully implemented, partly due to the immature compiler technologies of the time, but it did appear in restricted forms on the Atlas and Titan computers at Cambridge (Barron, Buxton, Hartley, Nixon & Strachey 1963).

CPL was heavily influenced by ALGOL 60, but unlike ALGOL 60 which was extremely small, elegant and simple, CPL was big, only moderately elegant and complex. It was strongly typed, but also had a "general" type enabling a weak form on polymorphism. It also had a purely functional subset, providing a `where` form of local definitions. List structures were polymorphic, with list selection being done through structure matching (Emery 1986). It was intended to be good for both scientific programming (in the way of Fortran and ALGOL) and also commercial programming (in the way of Cobol), but essentially tried to do too much. It could be seen as a similar effort to PL/1 in this way, or to later efforts such as Ada. However, the principle motivation for CPL was the contention that while ALGOL 60 was an ideal medium for defining algorithms, it was too far removed from the realities of computing hardware. It is probably this ideal more than anything else that led to the success of CPL as the progenitor of system programming languages, which are nothing if not efficient in the generation of object code. However, there was still a need for an efficient system programming language that was based along the original ideas of CPL but removed the layers of complexity and could handle the technology of the time. This was the prime motivator for the development of BCPL.

2.1.2 BCPL

BCPL (Basic Combined Programming Language) was designed as a seriously simplified, cut-down version of CPL. It was low-level, typeless and block-structured and was devised by Martin Richards at Cambridge University in 1966. The first compiler implementation was written while he was visiting MIT in the spring of 1967, with the language first described in a paper presented to the 1969 AFIPS[1] Spring Joint Computer Conference (Richards 1969).

[1] American Federation of Information Processing Societies

The language itself was lean, powerful, and portable. It proved possible to write small and simple compilers for it and was therefore a popular choice for bootstrapping a system. Reputedly some compilers could be run in 16Kb. Several operating systems were written partially or wholly in BCPL (for example, Tripos and AmigaDOS). A major cause of its portability lay in the form of the compiler, which was split into two parts. The front end parsed the source and generated OCODE, an assembly language for a stack-based virtual machine, as the intermediate language. The back end took the OCODE and translated it into the code for the target machine (Richards 1971). Soon afterwards this became fairly common practice, *cf.* Pascal PCODE or the Java Virtual Machine, but the Richards BCPL compiler was the first to define a virtual machine for this purpose. The most recent machine independent implementation of BCPL uses INTCODE, a low-level language interpreted by a fast and simple abstract machine (Richards 1975). This implementation can be downloaded free of charge (for private and academic purposes) from Martin Richard's personal website at the University of Cambridge (Richards 2000).

As stated previously, BCPL was typeless in the modern sense, but its one data type was the word, a fixed number of bits usually chosen to align with the architecture's machine word. It could be said that BCPL is an operator-typed language, rather than a declaration-typed language; that is, each object in the language can be viewed as having any type, and can change type at any time. It is the operators used to manipulate the object that determine the type it has at that moment. For example, `+` adds two values together treating them as integers; `!` indirected through a value effectively treats it as a pointer. In order for this to work, the implementation provided no type checking. This had obvious speed benefits from a compiler point of view, but created potential semantic problems. Other interesting properties of the language were all parameters were passed call-by-value and only one-dimensional arrays were provided. The philosophy of BCPL can summarised by quoting Martin Richards from (Richards 1967):

> *"The philosophy of BCPL is not one of the tyrant who thinks he knows best and lays down the law on what is and what is not allowed; rather, BCPL acts more as a servant offering his services to the best of his ability without complaint, even when confronted with apparent nonsense. The programmer is always assumed to know what he is doing and is not hemmed in by petty restrictions."*

This was a common philosophy with certain language paradigms and perhaps contributed to the open source ideal – the fact that the programmer has nearly total control over what the program can do, even if this means they can perform "dangerous" operations. This certainly helped BCPL gain large support, especially at a time when restrictive languages were becoming more commonplace (Emery 1986).

BCPL's heyday was perhaps from the mid 1970s to the mid 1980s, where implementations existed for around 25 architectures. Today it sees little use outside of legacy systems and academia. However, it is a useful language for experimenting with algorithms and for research in optimising compilers. Its descendant, C, is now the language of choice for system programming. The design of BCPL strongly influenced B, which developed into C. It is also said to be the language in which the original "`hello world`" program was written.

2.1.3 Development of B and C

As stated previously, the development of BCPL has a major impact on the design of system programming languages during the late 1960s and early 1970s. B was designed by Dennis Ritchie and Ken Thompson for primarily non-numeric applications and systems programming (Johnson & Kernighan 1973). It was essentially BCPL stripped of anything Thompson felt he could do without, in order to fit it on very small computers, and with some changes to suit Thompson's tastes (mostly along the lines of reducing the number of non-whitespace characters in a typical program). All B programs consisted of one or more functions, which were similar to the functions and subroutines of a Fortran program, or the procedures of PL/1. B, like BCPL, was a typeless language, all manipulations being on the implementation-dependent word (Emery 1986).

As in BCPL, the only compound object permitted was a one-dimensional vector of words. B did however, incorporate floating point operations, which was implemented in an untyped manner by providing a set of distinct operators that perform specialised operations on which are in effect short word vectors. However, apart from the typeless characteristics and the implications that follow from that, B

has more in common with C than with BCPL. The conventions of its syntax are similar to those of C; even extending to the use of identical symbols for identical purposes. Its runtime library foreshadows many of the features of the C library, and in common with many other languages, B was designed for separate compilation (Emery 1986). It achieved this aim not with the aid of a global vector like BCPL, but in the conventional way of linking at compile time like C. A program in B consists of a set of modules, which was not necessarily an individual file, but a distinct object presented to the compiler. Thus several modules could reside in a single file.

Early implementations of B were for the DEC PDP-7 and PDP-11 minicomputers running early UNIX, but as it was ported to newer machines, changes were made to the language while the compiler was converted to produce machine code. Most notable was the addition of data typing for variables. During 1971 and 1972 this became "New B" and eventually evolved into C. B was, according to Ken Thompson, greatly influenced by BCPL, but the name B had nothing to do with BCPL. B was in fact a revision of an earlier language, bon, named after Ken Thompson's wife, Bonnie (Ritchie 1993).

The C programming language came into being in the years 1969–1973 in parallel with the early development of the UNIX operating system (Ritchie & Thompson 1974). In contrast to BCPL and B, C is a typed language. This involved a totally different approach to compilation, with the introduction of a preprocessor. This is a simple macro processor that runs before the main compiler and was chiefly for handling named constants. Since it is a *pre*processor, it cannot interfere with the compiler, but it converts the program text into a form suitable for the compiler (for example, removing the readability introduced to aid human programming). Another spate of changes occurred between 1977–1979, when the portability of the UNIX system was being demonstrated (Johnson & Ritchie 1978). In the middle of this second period, the first widely available description of the language appeared: The C Programming Language, often called the "white book" or "K&R" (Kernighan & Ritchie 1978). However, in 1989, the language was officially standardised by the ANSI[2] X3J11 committee. In 1990, the ANSI C standard (with a few minor modifications) was adopted by the International Standards Organisation (ISO) as ISO/IEC 9899, with further modifications made in 1999 (ISO 1999).

Until the early 1980s, although compilers existed for a variety of machine architectures and operating systems, the language was almost exclusively associated with UNIX. More recently, its use has spread much more widely and today it is among the languages most commonly used throughout the computer industry (Ritchie 1993). C has a richer set of operators than any other widely used programming languages, with the possible exception of APL. You will find that in most cases C will do with an expression exactly what you intended to do. However, it is possible to write extremely unreadable C code, as shown by examples from the International Obfuscated C Code Contest (IOCCC 2003).

2.1.4 Code Examples

For simple code examples of BCPL, B and C, see Appendix B.

2.2 Compiler Phases

Since this is a discussion of developing a compiler for BCPL, it is important to address the issues with developing a compiler and what this entails. To do this, we need to understand the different stages in the compilation process and what actions occurs at each stage.

In simple terms, a compiler is a program that reads a program written in one language – the source language – and translates it into an equivalent program in another language – the target language (Aho, Sethi & Ullmann 1986). This translation could be to machine language, while some compilers output assembly language which is then converted to machine language by a separate assembler. A compiler is distinguishable from an assembler by the fact that each input statement does not, in general, correspond to a single machine instruction or fixed sequence of instructions. There are generally two parts to compilation: analysis and synthesis. The analysis part breaks up the source program into constituent pieces and creates an intermediate representation of the source program. The synthesis part constructs the desired target program from the intermediate representation. Of the two parts, synthesis requires

[2]The American National Standards Institute (ANSI) is a private, non-profit organisation that produces industrial standards in the United States.

the most specialised techniques.

As stated in (Aho et al. 1986), a compiler operates in phases, each of which transforms the source program from one representation to another. A typical decomposition of this process can be found in Appendix A. The phases are often collected into a front and back end, though often a third section is described (the "middle" end). The front end consists of those phases, or parts of phases, that depend primarily on the source language and are largely independent of the target machine. These normally include the lexical and syntactic analysis, where the source code is broken into tokens and syntactic structures are identified. These are also known as scanning (or "lexing") and parsing. The creation of the symbol table data structure occurs during these stages (Wilhelm & Maurer 1995). Semantic analysis is done to recognise the meaning of the program code to prepare building intermediate representations. Scoping and type-checking occurs here (if relevant to source language), with most compilation errors occurring at this stage. One of the final tasks is the generation of the intermediate language to pass to the middle and back ends. The front end also includes the error handling for each of these phases.

The middle end is usually language independent, except for a few callback hooks into the front end. The primary purpose of the middle end is to convert trees (abstract syntax) to a specific intermediate representation, using various code generation strategies. At some stage there should be a mapping into machine instructions, so essentially code generation is done first, then optimisation. There is also some, but not much, optimisation of the trees before the intermediate code is generated. In certain compiler models, the middle end phases are shared between the front and back ends.

The back end includes those portions of the compiler that are dependant on the target architecture and generally do not depend on the source language, just the intermediate language. In the back end, we find aspects of the code optimisation phases and code generation, along with the necessary error handling and symbol table operations. Resourcing and storage decisions become important, such as deciding which variables to fit into registers (if appropriate) and memory and the selection and scheduling of appropriate machine instructions (Wilhelm & Maurer 1995).

It has become commonplace to take the front end of a compiler and redo its associated back end to produce a compiler for the same source language on a different architecture. Alternatively, it is also possible to change the front end for a compiler and reuse the back end to compile a new language for the same machine (known as "re-sourcing"). For a more detailed discussion of the different phases of a compiler, see (Aho et al. 1986).

2.3 Static Single Assignment Form

Static Single Assignment (SSA) form is an intermediate representation (IR) developed by researchers at IBM in the 1980s in which every variable is assigned exactly once (Cytron, Ferrante, Rosenn, Wegman & Zadeck 1991). Existing variables in the original intermediate representation are split into versions, new variables typically indicated by the original name with a subscript, so that every definition gets its own version.

The primary benefit of SSA comes from how it simultaneously simplifies and improves the results of a range of compiler optimisations by simplifying the properties of variables. For example, consider the first three lines of code in Figure 2.1.

$$
\begin{aligned}
y &:= 1 \\
y &:= 2 \\
x &:= y \\
&\ldots \\
y_1 &:= 1 \\
y_2 &:= 2 \\
x_1 &:= y_2
\end{aligned}
$$

Figure 2.1: Simple SSA form example

We can see that the first assignment is unnecessary and that the value of `y` being used in the third line comes from the second assignment of `y`. A program would have to perform reaching definition analysis (a special type of data flow analysis) to determine this (Cytron et al. 1991). But if the program were in SSA form, as in the last three lines of Figure 2.1, both of these are immediate.

The increased potential for optimisation is the prime motivation for SSA, as many optimisation algorithms need to find all use-sites for each definition and all definition-sites for each use (for example, constant propagation must refer to the definition-site of the unique reaching definition). Usually information connecting all use-sites to corresponding definition-sites can be stored as *def-use* chains (where for each definition `d`, all pointers to all uses of `r` that `d` reaches are listed) and/or *use-def* chains (where for each use `u`, pointers to all definitions of `r` that reach `u` are listed). The improvement of def-use chains is one of the key feature of SSA, as each temporary has only one definition in the program, meaning that for each use `u` of `r`, only one definition of `r` reaches `u` (Novillo 2003*a*). Less space is required to represent def-use chains, as they only require space proportional to uses multiplied by definitions for each variable. SSA also eliminates unnecessary relationships and creates potential register allocation optimisations. Building def-use chains costs quadratic space whereas SSA encodes def-use information in linear space (Cytron et al. 1991).

By using a SSA-based implementation, it is possible to obtain large optimisation performance benefits for the compiler. Compiler optimisation algorithms which are either permitted or strongly enhanced by the use of SSA include constant propagation, dead code elimination, global value numbering, partial redundancy elimination, register allocation and sparse conditional constant propagation (for technical details concerning these machine independent optimisations, see (Aho et al. 1986) and (Grune, Bal, Jacobs & Langendoen 2000)). Numerous studies have shown the benefit of SSA forms; their application and optimisations are still current research areas. A method for converting into and using the SSA form is discussed in (Alpern, Wegman & Zadeck 1988).

2.4 GNU Compiler Collection Overview

GCC is the GNU[3] Compiler Collection. Originally, it stood for the "GNU C Compiler", but it now handles a range of different programming languages along with C. GCC is a GPL[4]-licensed compiler distributed by the Free Software Foundation[5] and a key enabling technology for the Open Source Software (OSS) and free software movements. Originally written by Richard Stallman in 1987, the main goal of GCC was to provide a good, fast compiler for machines in the class that the GNU system aims to run on: 32-bit machines that address 8-bit bytes and have several general registers. Elegance, theoretical power and simplicity became only secondary considerations in the design (Stallman 2004*b*).

An important design decision for GCC has been its high level of reuse of the larger part of the source code. A key goal was for the compiler to be as machine independent as possible and GCC managed this by getting most of the information about target machines from machine descriptions which give an algebraic formula for each of the machine's instructions. Therefore, GCC does not contain machine dependent code, but it does contain code that depends on machine parameters such as endianness and the availability of auto-increment addressing. The purpose of portability is to reduce the total work needed on the compiler (Stallman 2004*a*). GCC is now maintained by a varied group of open source programmers from around the world, and has been ported to more kinds of architectures and operating systems than any other compiler. GCC has been adopted as the main compiler used to build and develop for a number of systems, including GNU/Linux and Mac OS. The current release series (as of 2004-04-20) is GCC 3.4.0, which is available from the GCC Home Page (GCC 2004*c*). For further details about the current development of GCC, see (GCC 1999). A schematic of the existing GCC infrastructure is shown by Figure E.1 in Appendix E.

2.4.1 GCC Front Ends

As described in Section 2.2, the front end of a compiler handles the source language-dependent tasks and is largely independent of the target machine. The interface to front ends for languages in GCC,

[3]The GNU Project was launched by Richard Stallman on 27th September 1983 with the goal of creating a free operating system. GNU is a recursive acronym for "GNU's Not UNIX". See `http://www.gnu.org` for further details.

[4]The GNU General Public License is a copyleft free software license.

[5]A tax-exempt charity founded by Stallman in 1985 to provide logistical, legal and financial support for the GNU Project. See `http://www.fsf.org` for further details.

and in particular the abstract syntax tree structure, was initially designed for C, with many aspects of it still somewhat biased towards C and C-like languages. The official GCC distribution currently contains front ends for C (*gcc*), C++ (*g++*), Objective-C (also *gcc*), Fortran (*gfortran*), Java (*GCJ*) and Ada (*GNAT*). Several front ends exist for GCC that have been written for languages yet to be integrated into the official distribution, including the GNU Pascal Compiler (*GPC*), Cobol for GCC, GNU Modula-2 and GHDL (VHDL front end). Therefore, by developing a new front end for GCC, it is possible to create a compiler that can produce machine code for approximately thirty processor families, including Intel x86, Sun SPARC, DEC Alpha, ARM and Motorola 68000.

2.4.2 GCC Intermediate Representations

Register Transfer Language (RTL) is an intermediate representation (IR) used by the GCC compiler and is used to represent the code being generated, in a form closer to assembly language than to the high level languages which GCC compiles. RTL is generated from the language-specific GCC abstract syntax tree representation, transformed by various passes in the GCC middle end, and then converted to assembly language (Stallman 2004*a*). RTL is usually written in a form which looks like a Lisp S-expression[6]. For example, the "side-effect expression" shown in Figure 2.2 says "add register 138 to register 139, and store the result in register 140".

$$(set : SI(reg : SI140)(plus : SI(reg : SI138)(reg : SI139)))$$

Figure 2.2: Example RTL register S-expression

The RTL generated for a program is different when GCC generates code for different architectures, though the meaning of the RTL is more or less independent of the target: it would be usually be possible to read and understand a piece of RTL without knowing for which processor it had been generated. Similarly, the meaning of the RTL does not usually depend on the original high-level language of the program (Stallman 2004*a*). However, recent developments with GCC have introduced a new intermediate representation framework to improve optimisations and language independence.

The goal of the GCC SSA for Trees project is to build an optimisation framework for abstract syntax trees based on the SSA form (see Section 2.3). While GCC trees contain sufficient information for implementing SSA, there exists two major problems. Firstly, there is no single tree representation in GCC, as each front end defines and uses its own trees. Secondly, the trees are arbitrarily complex, giving problems for optimisations that wish to examine them, along with code duplication in the numerous versions.

To address the first problem, GCC have developed a common tree representation called GENERIC that is able to represent all the constructs needed by the different front ends whilst removing all the language dependencies (GCC 2003*c*). The design of GENERIC was first discussed on the GCC mailing lists due to fundamental flaws with the existing tree intermediate representations as it was felt that they were designed as a convenient shorthand for representing C trees, but is fairly unwieldy for use by optimisers (Merill 2002*a*). Essentially, it seemed as if the current tree intermediate representation was restricting the amount of possible optimisations, as even some simple optimisations were not being fully implemented. In the `tree-ssa` branch, sparse conditional constant propagation, dead code elimination and partial redundancy elimination with strength reduction (among others) have been implemented in the optimisation passes. The original discussion of the design can be found on the GCC mailing lists (Merill 2002*b*).

To address the complexity problem, GCC have implemented a new simplified intermediate representation based on GENERIC. The intermediate representation, called GIMPLE, is a simple C-like three-address language that is straightforward to analyse and maintains all of the high-level attributes of data types. GIMPLE is derived from the SIMPLE representation developed by the McCAT project from McGill University (Hendren, Donawa, Emami, Gao & Sridharan 1992).

[6] An S-expression is a convention for representing data or an expression in a computer program in a text form. The most common feature is the extensive use of prefix notation with explicit use of brackets.

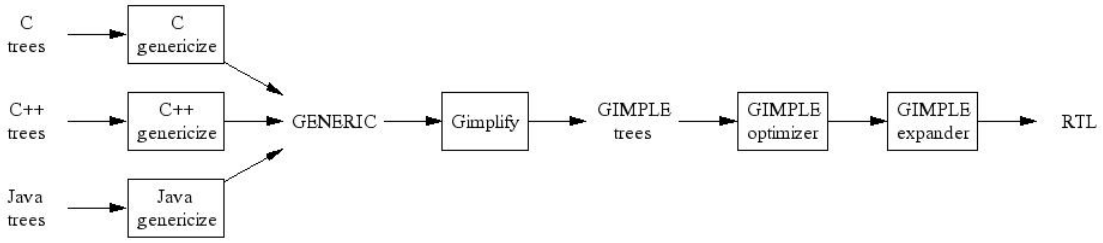


Figure 2.3: GCC implementation of SSA (Novillo 2003*a*)

After nearly two years of work, the `tree-ssa` side branch was fully merged into mainline (2004-05-12) and is now the active development branch for the next GCC release, 3.5.0. The stabilisation process has being driven by the merge criteria defined in (GCC 2003*c*), with current API documentation (for example, listing data structures, source file lists, data fields, globals) and an updated version of the GCC Internals Manual (Stallman 2004*a*) available.

2.5 Possible Implementations

The following sections discuss a range of different approaches for the project implementation.

2.5.1 Flex and Bison

Lex (Lexical Analyser Generator) and Yacc (Yet Another Compiler Compiler) are tools designed for generating compilers and interpreters and were both developed at Bell Laboratories in the 1970s. Both Lex (Lesk & Schmidt 1975) and Yacc (Johnson 1975) have been standard UNIX utilities since 7th edition UNIX (circa 1978), but most UNIX or GNU/Linux systems use the GNU Project's distributions of Flex (Fast Lexical Analyser Generator) and Bison (Yacc-compatible parser generator). Flex is a tool for generating programs that perform pattern-matching on text, and is mainly used for developing compilers in conjunction with GNU Bison. Bison is a general-purpose parser generator that converts a grammar description for an LALR context-free grammar (a Chomsky Type II grammar, see (Chomsky 1956) for further elaboration) into a C program to parse that grammar. For this project, we will be using the GNU tools, but the terms Lex/Flex and Yacc/Bison are often used interchangeably.

The starting point to this problem would be to use Flex and Bison to create a scanner and a parser to process the BCPL code, which could then be compiled using an appropriate C compiler. The hardest part is ensuring that the lexical, syntactic and semantic phases are robust enough to cope with the intricacies of the BCPL syntax (see Section 2.6.1 for further details). This phase of the development could be time consuming, due to the complexity of certain language features in BCPL (see Section 2.6). However, this would be common starting point for all potential approaches, as each would require some form of lexical and syntactic phase. Certain problems can arise with using these tools, more so if you wish to customise the generated parser, as it is generally accepted that Bison produces poor C code for humans to maintain.

2.5.2 BCPL to C Conversion

The process of converting a high-level language into another could be thought of as a somewhat loose definition of compilation. In the case of BCPL to C, it could be thought of as a form of "up-compilation", due to the relationship between BCPL and its descendant C. Nevertheless, difficulties occur when trying to convert whole programs into a different programming language when certain language features are not duplicated, or when certain semantics are hard to preserve. For example, converting from a functional language such as Haskell to an imperative language like C is certainly not trivial, with handling variable assignment and preserving referential transparency being two obvious problems. However, as discussed in Section 2.1.3, C evolved from B, which itself developed from BCPL. This means that certain language features in C have persisted or have derived from features available in BCPL, making it a reasonable choice as an intermediate language. It would not be possible, however, to do a straight conversion, as there are some major differences in some critical areas. An obvious example would be handling the conversion into data types, which would have to be done very carefully as it would require analysis of the surrounding environment and operators. It would be far too crude to

attempt to convert everything to a single type, such as an integer. This is a perennial problem with the acquisition of type information from a typeless language; a common example would be the conversion of JavaScript code. A further problem would involve ensuring that consecutive words in memory have consecutive numerical values (especially with respect to pointer arithmetic) and also the problems of communicating with the global vector (discussed further in Section 5.6.8).

Previous work does exist in this area (see Section 2.5.8), so it may be possible to build upon this and extend the existing functionality, especially with the handling of the BCPL system library. One of the downsides with this approach is with the inelegance of the solution, as it would seem crude to force BCPL to C and then compile as normal with a C compiler. Also, as C's runtime semantics do not make it easy to honour BCPL's guarantees about dynamic variable values, it would be by no means a trivial solution. If this were the chosen approach, it would probably be much easier to use a scripting language such as Perl to write a converter between BCPL and C, but this would circumvent some of the requirements of the project.

2.5.3 Norcroft C Compiler

This second approach builds on the first approach by using Flex and Bison, but instead uses the Norcroft[7] C compiler (NCC) middle and back end phases. The Norcroft C compiler is a optimising and re-targetable ANSI C compiler, which includes many lint-like features and warnings for common syntactic and semantic errors (Mycroft & Norman 1986). This approach would entail re-sourcing the existing front end and building Norcroft intermediate trees for BCPL. We would then be able to reuse some of the existing tree optimisations and generate code with the back end. The Norcroft compiler is a robust and proven C compiler, but the main problem is that it lacks full documentation with respect to functionality, interfaces and structures. It was not developed with re-sourcing in mind; this is highlighted by some C language-specific functionality in the middle and back ends. This in itself is not a major problem, but falls short in comparison to other strategies discussed in the following sections. Previous work also exists this this area, as this strategy was attempted in an undergraduate project in 2003 with some success (see Section 2.5.8 for further details).

2.5.4 lcc, A Re-targetable Compiler for ANSI C

lcc is a re-targetable compiler for ANSI C developed by the Department of Computer Science at Princeton University. It generates code for the ALPHA, SPARC, MIPS R3000 and Intel x86 architectures. It has a small and compact interface that includes only certain essential features, but which make certain simplifying assumptions that limits the interface's applicability to other languages (Fraser & Hanson 1991*a*). However, many of these assumptions concern the relative size and properties of different data types; this may not be a problem for the untyped BCPL language. Nevertheless, it is still hard to anticipate the range of restrictions that could exist within the infrastructure.

The lcc approach is similar to using the Norcroft compiler, as we would replace the existing front end and reuse the existing middle and back ends. The current lcc front end performs lexical and syntactic analysis as normal, with some minor optimisations. This is connected to the back end by an intermediate language consisting of 36 operators and shared data structures. However, the front and back ends are tightly coupled, with the functions in the interface unevenly split between each to enable callback (Fraser & Hanson 1991*b*). The major disadvantage of lcc is that it does no further optimisation other than what is done in the front end. This disregards a whole range of optimisations that are more effective when performed at the intermediate code level. Also, because of the seemingly poor modularity between the front and back ends, replacing the early phases would not be trivial and could potentially require removal of the optimisation algorithms. Nevertheless, the popularity and pedigree of this production compiler cannot be ignored, but it would require a larger amount of work even when compared with using NCC. Further information about lcc can be found on the project website (Fraser & Hanson 2003).

2.5.5 The Stanford University Intermediate Format Project

The SUIF (Stanford University Intermediate Format) compiler, developed by the Stanford Compiler Group, is a free infrastructure designed to support collaborative research in optimising and parallelis-

[7] Available from Codemist Ltd. See `http://homepage.ntlworld.com/codemist/ncc/` for more information.

ing compilers. It is a part of the national compiler infrastructure project funded by DARPA[8] and the NSF[9] (Aigner, Diwan, Heine, Lam, Moore, Murphy & Sapuntakis 2003). One of the project's main aims has been to create an extensible and easy to use compiler infrastructure but some efficiency and robustness may have been sacrificed to achieve this (SUI 2001). The compiler is based upon a program representation, also called SUIF, with the emphasis on maximising code reuse by providing useful abstractions and frameworks for developing new compiler passes. It also provides an environment that allows compiler passes to easily inter-operate (Aigner et al. 2003).

The SUIF2 system is a new design and implementation and is very different from the previous SUIF1 system, which was originally designed to support high-level program analysis of C and Fortran programs (Aigner et al. 2003). In comparison to the lcc infrastructure, SUIF has a well designed, modular subsystem that allows different components to be combined easily. It also has an extensible program representation that allows users to create new instructions to capture new programs semantics or new program analysis concepts. The existing predefined object hierarchy can thus be further extended or refined to create news abstractions (Aigner et al. 2003). This would seem to be suitable for our needs, though it is not clear how easy it would be to re-source and redefine the front end and also assemble a suitable back end from existing passes. However, the modular nature of this infrastructure and the very fact that it was designed to be extensible means SUIF is an extremely viable option; more so with the fact that the user is insulated from the details of the implementation (by using a high-level specification language called "hoof" that is macro translated into C++ interfaces and implementations). As remarked for the lcc project, the popularity and pedigree of this compiler framework cannot be ignored, but it is hard to gauge the amount of development required to re-source a SUIF implementation. Further information about SUIF can be found on the project website (SUI 2001).

2.5.6 The Low Level Virtual Machine Project

The LLVM (Low Level Virtual Machine) compiler infrastructure project is a product of the Lifelong Code Optimisation Project in the Department of Computer Science at the University of Illinois, Urbana-Champaign. Like SUIF, it is sponsored by both the NSF and DARPA. Fundamentally, LLVM is a compilation strategy designed to enable effective program optimisation at compile time, link time, runtime and offline, while remaining transparent to developers. LLVM's virtual instruction set is a low-level code representation that uses simple RISC-like instructions. However, it provides rich, language-independent, type information and data flow (SSA) information about operands (Lattner & Adve 2004*b*). LLVM is also a collection of source code that implements the language and compilation strategy. The primary components of the LLVM infrastructure are:

- A GCC-based C and C++ front end.
- A link-time optimisation framework.
- Static back ends for the SPARC v9 and Intel x86 architectures.
- A back end which emits portable C code.
- A Just-In-Time (JIT) compiler for SPARC v9 and Intel x86 (Lattner & Adve 2004*a*).

LLVM is a robust system and is particularly well suited for front end development, as it currently includes front ends for C, C++ and Stacker (a Forth-like language), with front ends for Java, Microsoft CLI and Objective-Caml in early development. LLVM would definitely be suitable for this project, especially with the extensive online developers' documentation (for example (Lattner, Dhurjati & Stanley 2004) and (Lattner & Adve 2004*b*)). One downside is the fact that it is implemented in C++, with heavy use of the STL (Standard Template Library). A lack of technical expertise with C++ would certainly make re-sourcing LLVM harder, more so with the heavy use of templates. Nevertheless, the aggressive life-long optimisation model of LLVM, along with the range of targets and native code generators would certainly make it a viable infrastructure for this project. It has become a popular and commonly-used compiler infrastructure and is still under active development. Further information about LLVM can be found on the project website (Lattner 2004).

[8]The Defence Advanced Research Projects Agency (DARPA) is the central research and development organisation for the US Department of Defence.

[9]The National Science Foundation (NSF) is an independent US government agency responsible for promoting science and engineering through programs that invest in research and education.

2.5.7 GNU Compiler Collection

The popularity and pedigree of GCC is arguably the best out of the options discussed in this section. As mentioned previously in Section 2.4, the officially supported front ends are complemented by a number of front ends in active development. GCC has been available since 1987, when the v0.9 beta was first released. The years of development that have followed have established GCC as the de facto open source C compiler. Detailed documentation exists for working with GCC (Stallman 2004*b*) and developing for GCC (Stallman 2004*a*). There also exists a wide range of information about developing front ends for GCC (GCC 2003*b*), including a "Toy" language example and also contributions from existing front ends (see Section 2.5.8).

Developing a front end for GCC, rather than compiling directly to assembler or generating C code which is then compiled by GCC, has several advantages. For example, GCC front ends benefit from the support structure for many different target machines already present in GCC, along with the range of optimisations. Some of these, such as alias analysis, may work better when GCC is compiling directly from source code then when it is compiling from generated C code (Stallman 2004*a*). This is an important point, as better debugging information is generated when compiling directly from source code than via intermediate generated C code. As discussed previously, GCC has developed significantly from being just a C compiler. The framework exists to enable the compilation of languages fundamentally different from those for which GCC was designed, such as the declarative logic/functional language Mercury (GCC 2003*b*). It is hard to create a truly independent and all-encompassing compilation strategy with respect to intermediate representations and data structures, but the aims of GCC have been to develop a language-independent tree representation and a large number of optimisation algorithms that work on it.

As discussed more thoroughly in Section 2.4.2, the incorporation of the two new high-level intermediate languages (GENERIC and GIMPLE) and the optimisation framework for GIMPLE based on the SSA form for the GCC 3.5.0 release has created an opportunity to develop a cutting-edge front end using the future GCC internal structure. The various improvements to the internal structure of the compiler, along with the several SSA-based optimisers have created new optimisation opportunities that were previously difficult to implement. A problem exists with the frequency of up-to-date documentation available for GENERIC and GIMPLE, as updates to the GCC Internals manual (Stallman 2004*a*) are still incomplete. It has been remarked that if you can describe it with the codes in the source definition file `gcc/tree.def`, it is in GENERIC form (Merrill 2003). It is not an ideal situation to develop in a language that is not fully documented, but the existing front ends should provide good starting points. Due to the relationship to C, it would feasible to develop from the C front end and attempt to reuse as much as possible for a BCPL front end. The sensible approach would be to have the parser directly build GENERIC trees, as this has the advantage that we would not need to define our own language-specific syntax trees and then face problems in the conversion to GENERIC form.

Another issue to be considered is the scale of the proposed task. This would be the similar for many of the options discussed here, but developing a GCC front end is known to be a difficult but elegant solution. Therefore, the amount of time allocated for this project would mean that it is unlikely that a complete working compiler is feasible. For most options, and especially GCC, it is likely that the latter stages of the project would be more research based and become a detailed discussion about the potential implementation. Nevertheless, it seems that GCC would be ideal for this project, especially from a research and educational perspective. Further information about GCC and its releases can be found on the project website (GCC 2004*c*).

2.5.8 Existing Work

For what is remarked to be an popular, but unused, legacy system programming language, numerous attempts at developing modern compilation strategies for BCPL exist. This section is a discussion of existing implementations and recent developments involving BCPL.

- Martin Richard's original BCPL compiler implementation has been updated and ported to most modern architectures, as mentioned in Section 2.1.2. It uses INTCODE, an low-level language interpreted by a fast and simple abstract machine (Richards 1975). This would be an obvious resource for understanding how the original implementation of BCPL was done and also how the designer of the language envisaged its modern implementation. It also seems that the BCPL to C conversion route has been done previously, with numerous levels of success. However, after

consultation with Martin Richards via email, he was unaware of implementations that have re-sourced an existing compiler tool, especially not with GCC. He has also developed some BCPL to C conversion routines and has previously entertained the idea of developing a GCC front end, but little has been done in recent years. Further information about Martin Richard and his BCPL implementation can be found on his website at the University of Cambridge (Richards 2000).

- Mark Manning of Selwyn College, University of Cambridge, has also previously worked on a compiler for BCPL. His implementation uses the BCPL to C conversion approach, which is then compiled by GCC. This is, therefore, not a true front end for GCC as claimed on his website (Manning 1999), but a converter for BCPL to ANSI C using certain GCC extensions. However, it was not designed as a true C converter, due to the intermediate stages producing C code that would not be written by a human programmer. The code produced is merely a stage before compilation via GCC. The program implements the BCPL language as defined in (Middleton, Richards, Firth & Willers 1982), along with some commonly-used extensions such as the infix byte operator. There is some promising work that could be reused, especially with the implementation of the BCPL system library and the global vector in C. As discussed previously, the translation of BCPL to C is by no means trivial and this is highlighted by the fact that `LONGJUMP` and `LEVEL` have yet to be implemented, along with a range of other library functions. Further information about Mark Manning and his work can be found on his website at the University of Cambridge (Manning 1999).

- Fergus Henderson, formerly of the Department of Computer Science and Software Engineering at the University of Melbourne, has developed an example front end for GCC based around a simple "Toy" language[10]. The Toy language is a simple language intended to demonstrate some of the basic concepts in developing a language front end for GCC. The Bison grammar contains a small amount of productions, with the scanner handling a small subset of keywords and tokens. The most recent version was designed to work with GCC 2.95 and 3.0, so the generation of the intermediate representation code will not be relevant to building GENERIC trees. However, the Toy language is a good starting point of how to develop a front end for GCC and could be useful for creating the appropriate configuration and build files. It will also be simpler than looking at one of the official release front ends, due to the primitive nature of the language. A similar example front end, including a GCC front end "How-To" has also been written by Sreejith Menon, of the Department of Computer Science and Automation at the Indian Institute of Science[11].

- Developing a compiler for BCPL has previously been set as a undergraduate dissertation project at the University of Bath. In 2003, it was undertaken by Darren Page, who attempted to re-source the Norcroft back end. This was met with some success, with the Norcroft compiler proving itself to be robust enough to handle most of the BCPL language. Some of BCPL's more troublesome features such as `VALOF` and `RESULTIS` had similar representations in the Norcroft intermediate data structures. However, he noted problems with using the Norcroft compiler due to its lack of documentation, especially with using the Norcroft abstract syntax tree as the intermediate representation (Page 2003). It may be possible to reuse some of the existing infrastructure, as Lex and Yacc were used for the early phases. It might also be possible to develop upon the implementation of some of the more difficult language features with respect to building the intermediate representations.

- As would be expected, the official GCC Front Ends resource (GCC 2003*b*) is a very good repository for information about developing a front end for GCC. It describes the main front ends distributed with the official releases of GCC, but also details existing front ends that have yet to be integrated into the main distribution. A further resource is an overview of front ends that are currently works in progress, enabling comparisons with how they are implemented and how certain language features are handled. Some of the more advanced front end resources include the GNU Pascal Compiler (GNU 2003*c*); GNU Modula-2 (GNU 2004*b*); COBOL for GCC (Josling 2001); PL/1 for GCC (Sorensen 2004); and the Design and Implementation of the GNU INSEL Compiler (Pizka 1997).

There may be some difficulties in analysing and reusing some of the existing work, but much of it represents good starting areas on how to handle BCPL language features or how to start developing a

[10] The Toy language is available for download from `http://www.cs.mu.oz.au/~fjh/gcc/example-front-end.shar`.
[11] Available from `http://people.csa.iisc.ernet.in/sreejith/`.

front end for GCC. Both Martin Richards and Mark Manning have been good sources of information for their work and have expressed an interest in the development of this project.

2.6 Design Issues

2.6.1 BCPL Syntax Features

The relationship between BCPL and C was discussed in Section 2.1.3, but there are certain language features in BCPL that raise potential problems for this project. Developing a compiler for BCPL will involve the consideration of many similar issues to writing a compiler for other procedural languages that have static scoping. For example, if expressions were confined to those conventionally found in other languages, then functions in BCPL would be very restrictive indeed, rather like statement functions in Fortran (Emery 1986). However, BCPL provides the operator `VALOF` which enables a function to be defined in terms of a sequence of commands. A `VALOF` operator must always be matched with a corresponding `RESULTIS` command within the block (Emery 1986). Implementing the `VALOF` operator will not be trivial, as it may require the declaration of a function for each `VALOF` block. Alternatively, a solution may exist using a GCC extension, which would be a problem if ANSI C-compliance was a requirement.

Another example, is that in contrast to most other block-structured languages, BCPL permits a direct jump from an inner block to an outer block containing it, i.e. from a higher to a lower level on the stack (Richards & Whitby-Strevens 1980). Such a jump is typically performed in the case of exceptions. By providing a direct jump out of a block, rather than obliging control to retrace the full sequence of calls that got it there, BCPL in some sense "saves" code. A jump out of a block involves two actions: going to a label in another block, and restoring the stack to a level appropriate for the code following that label (Emery 1986). The jump is performed by using the library routine `LONGJUMP`, with the destination label having been declared globally. `LONGJUMP` and `LEVEL` will be troublesome features to implement, because it is unknown how the the required scoping and stack functionality will be implemented using GENERIC.

Some of the other issues that need to be considered are the passing of call-by-value parameters, block/lexical scoping, creation and destruction of stack frames and the implementation of the global vector. Also, since no official standard for the language was ever completed, the range of commonly-used extensions from the different implementations of BCPL may conflict or be difficult to implement.

2.6.2 Back End Phases

The back end processes are arguably the most complicated to implement in a compiler, due to the intricacies involved with the code generation and optimisations for different platforms. This is one of the keys benefits from reusing an existing infrastructure, as it eliminates the need to provide code generation and optimisation phases and also enables targeting of the chosen language to the architectures supported by that infrastructure. For example, by using GCC, it would be possible to compile BCPL to run on over 30 architectures, including Intel x68, Sun SPARC and ARM. It is obvious from numerous posts and discussions on the GCC mailing lists that targeting to a new platform is by no means a trivial task, so to remove this from the project scope to concentrate on the front end is an easy decision. Also, the back end phases do not represent the main aims of this project, as we are more interested in providing a compiler framework for BCPL, rather than providing one for a particular architecture. The timescales for this project would also inhibit developing a complete back end for even a single architecture. This would preclude having to investigate and implement the range of machine independent and dependent optimisations (for example, common subexpression elimination and peepholing (Aho et al. 1986)), as this will be provided for by the GCC middle and back ends.

2.6.3 Coding Standards

The use of coding standards in this project will be important if any approach, but in particular the GCC approach, is used. For work to be accepted by the maintainers of GCC, both GNU and GCC have defined coding standards (GNU 2003*a*, GCC 2003*a*) which have to be adhered to so as to ensure consistency with existing code. Therefore, it will be necessary to research the defined coding styles and language features that are recommended or disallowed within the GCC code base. Some simple examples covered would be naming conventions, code formatting and Makefile conventions.

2.6.4 Legal

There also exists certain legal ramifications when developing for GCC or GNU. If this project reaches a stage where it is mature enough for a public release, it would be necessary to release, for example, the GCC front end for BCPL under the GNU Public License (GNU 2003*b*). In this case, certain prerequisites would have to be met, including assignments or disclaimers of copyright. This is not currently an issue, but needs to be considered, especially as there could be a conflict with the University of Bath's intellectual property regulations (see dissertation copyright declaration). For further details, see the Contributing to GCC website (GCC 2004*a*).

Chapter 3

Requirements

A well constructed set of requirements are essential to the success of a software development effort (Pressman 2000). With this in mind, the requirements specification below is based on the structure provided by the IEEE Recommended Practice for Software Requirements Specifications (IEEE 1998).

3.1 Requirements Analysis

The numerous methods for analysis can be categorised broadly into *top-down* and *bottom-up* approaches. Top-down approaches, such as a goal-directed method (Sommerville 2001) advocate the use of high-level goals as a starting point, whereas bottom-up methods, such as the production of use cases (Sommerville 2001) emphasise details of system usage scenarios. The approach taken for this project was to use a top-down, goal-directed approach.

3.2 Requirements Specification

This section identifies and discusses key requirements for the project and will focus upon areas of particular difficulty.

3.2.1 System Perspective

The aim of this project is to investigate re-sourcing GCC for the BCPL language. We will attempt to adhere to the BCPL language as defined by the Backus-Naur Form grammar as set out in Appendix C, which has been adapted from the BNF in (Richards & Whitby-Strevens 1980). We will endeavour to implement some of the commonly-used extensions in the language.

The intention is to develop a GCC front end for BCPL using the current active mainline development branch (previously the `tree-ssa` branch), which will be released as GCC 3.5.0 at some point in 2005. The implementation of the SSA-based optimisation framework will provide a host of performance benefits, especially with the integration of the two new high-level, language independent intermediate representations, GENERIC and GIMPLE (Novillo 2003*b*). The BCPL front end will be implemented as extension of GCC, using existing functionality such as the intermediate representations, optimisation passes and code generation phases.

3.2.2 System Features

The system requirements for this project would be similar to many other compiler requirements, though especially GCC. For example, the application should accept an input file of BCPL code. It should then perform lexical and syntactic analysis on this file; if any errors are detected at this stage, the user should be notified with appropriate error information and, depending on severity, the build should cease. After these stages, the system should build the GENERIC intermediate representation trees and then pass these to the GCC middle and back end. At this point, the compilation would become entirely controlled by the GCC process. The GENERIC representation would then be lowered into GIMPLE form ("gimplification") and the GCC optimisation and SSA passes would be run on it. If no errors are detected and the build completes successfully, then an executable should be produced, depending on the host and target platform. This would be the basic usage of the compiler, though

it should be possible at a later stage to replicate the same options available for compilation as with GCC. Examples of this would be halting after preprocessing, dumping the parse tree and producing assembler output. It should also be possible to produce linkable object files.

As previously discussed in Section 2.6.4, there may be certain legal issues affecting the system when developing for GCC. The ramifications of reassigning the copyright to the project as and when it is ready for public release is not entirely known, so would require further investigation. Nevertheless, this does not currently affect the development but would need to be considered at a later date.

Emergent system properties for this project may be hard to predict, but it is feasible to aim for a high level of reliability. Other factors like response time, storage capacity and constraints on input/output/throughput would be dependent on the host system hardware specification. It may be necessary to set a minimum hardware requirement for memory, hard drive capacity or even possibly processor speed. This is currently not done for GCC, aside from the minimum specification required for installation and as long as the hardware itself is still supported. Security factors are not relevant for this project, though on certain systems access requirements may apply. It will, however, be necessary to ensure that the compiler is tested thoroughly with problematic, high-resource and boundary cases so as to ensure a large coverage of potential problem areas. In this way, common application problems such as buffer under/overflows and memory leaks may be discovered. There may also exist non-functional requirements in the system such as constraints on the development process and the ramifications of relevant standards. However, a common problem with many non-functional requirements in a system is that they are often hard to verify (Sommerville 2001), such as an analysis of reliability.

Lexical Analyser

The lexical phase of the compiler should take the source language as input and break it into specific set of lexical tokens (Aho et al. 1986). These tokens should represent the units of information in the language, such as identifiers, keywords and operators. The lexical phase should control the input of the source code file and should ensure that the named file exists and is accessible. If the file is not found or is not accessible, an error should be produced. The lexical analyser should be produced using the GNU compiler tool Flex and should use the necessary language features (such as regular expressions, pattern matching and exclusive states) to produce an efficient lexical phase. It should also be developed so that easy integration with a Bison-generated parser is possible, ensuring any shared data structures are accessible. There should also exist some form of error detection, analysis and recovery in the lexical phase. This should notify the user with details about the type and location of the error and, depending on severity, halt compilation.

Symbol Table

The symbol table will be the most persistent data structure during the compilation process. It is important for it to provide quick access, to be resource efficient and available to all phases that require access. For the chosen approach, it is particularly important for the symbol table to be visible to the GCC middle and back ends, especially during the lowering stages from GENERIC to GIMPLE form and the optimisation passes. For a prototype compiler, it would not be necessary to implement a optimally efficient symbol table, but we have chosen to implement a separately-chained hash table for our symbol table (see Section 4.4 for technical details). At this time, the efficiency given by the hash table implementation would be appropriate for the scale of the project. This should give a scalable implementation that could be improved upon in a later version. The symbol table should provide basic functions to insert, lookup and delete entries in the symbol table. It should ensure that any resource allocations it makes are released when sensible (for example, on error or when the build completes). For diagnostic purposes, it should provide a function to dump the contents of the symbol table to a file. Any further functionality should be provided if and when required during development.

Parser

The parser should be implemented as a bottom-up parser generated by GNU Bison, and should be able to easily integrate with a lexical analyser produced by Flex. It should drive the lexical phase and parse the code from the selected input file. The implementation of the grammar productions in the Bison parser should follow the BCPL language as defined in the BNF grammar given in Appendix C. However,

during development these may require minor changes to resolve ambiguities and remove conflicts or errors. The parser should be able to access the symbol table functions to add, lookup or remove entries.

One of the main tasks of the parser is to start building the intermediate representation from the parsed input code. When a grammar production is completed and accepted, the parser should build the relevant tree nodes in GENERIC format. No other internal, bespoke or custom tree representation should be used, apart from the GENERIC intermediate language as defined in (Stallman 2004*a*). A key design feature of the parser is its integration into the GCC infrastructure and how the intermediate representations are passed onto the GCC middle and back ends. It should ensure that the GENERIC trees are passed successfully and that resources are released whenever possible. Like the lexical analyser, some error detection, analysis and recovery should be available in the parser and this again should notify the user with further information about the type and location of the error and, depending on severity, halt compilation. As discussed in the requirements for the symbol table, efficiency is often a factor in the parser and so attempts should be made to produce a fast and efficient parser with Bison.

Intermediate Representations

For most compiler implementations, the intermediate representation is invariably designed specifically for the language and reflects certain features of that language. However, since we have chosen to use the GCC framework in this project, it is hard to state or define requirements. This is partly due to the fact that we have no control over the development and direction of the language and also because it has yet to be conclusively defined and fully documented. This problem would certainly represent a non-functional requirement or an emergent system property, as it is not under the control of this project. This is mitigated by the recent developments in GCC that make it highly unlikely that major changes to either of these representations or their usage will occur. Nevertheless, we will adhere to the GENERIC and GIMPLE language features as detailed in (Stallman 2004*a*) and reuse existing front end code to ensure some level of conformity.

GCC Integration

The problems discussed above are similar for the integration with GCC as for the intermediate representations. Because we are utilising an existing compiler framework to handle all of the optimisation passes and code generation phases, it is difficult to define requirements for these phases. However, information describing certain parts of the integration of the existing front ends in GCC is given in (Stallman 2004*a*), but much is missing. It should also be possible to set some metrics about certain expected levels of performance. Without any optimisation options set, the compiler's goal is to reduce the cost of compilation and to make debugging produce the expected results (Stallman 2004*b*). The compiler should, however, be able to provide a range of optimisation flags that would attempt to improve the performance and/or code size at the expense of compilation time and possibly the ability to debug the program. As stated, we will have available the whole range of optimisation options for GCC (see (GCC 2004*d*) for further details). The requirements would also be constrained by the target architectures which are currently available for GCC. However, much of this area is beyond the scope of the project, but it is important to recognise the portability of GCC and the difficulty in porting GCC to a new architecture.

3.2.3 User Characteristics

The users of this application would typically be BCPL (or possibly C) programmers, experienced with using compilers and in particular, GCC. No special access or superuser privileges would normally be required, but this may differ from system to system. However, by the very nature of this application, the user should be an experienced computer user with a knowledge of system programming.

3.2.4 Operating Environment

This project was implemented and tested on the GNU/Linux operating system (see Appendix G for version information), so initially the operating system requirements would be related GNU/Linux systems. However, as the integration into GCC progresses, the range of operating environments would be the same as is for GCC (see (GCC 2004*c*) for further details). At first, the compiler would be delivered as a pre-compiled binary application, but if and when GCC integration was completed, it would be possible to distribute with the GCC source or as a pre-built GCC binary. In the case of

distributing as source, an existing compiler that is able to bootstrap GCC would then be required to build it. For further details of the programs used in the development of this project, see Appendix H.

3.2.5 Documentation

As previously mentioned, the users of this system would need to be proficient with the BCPL language (or a related language like C) and have a working knowledge of compilers. With this in mind, providing system documentation or a user manual was deemed not to be primary goal of the project. Nevertheless, the standard command line help (for example with GCC, which can be invoked on the command line by `gcc -h`), detailing the available compiler options will be provided, along with a version information option (again with GCC, `gcc --version`). Also, all of the source code should be commented, with important features and functionality explained (in accordance with the GNU and GCC coding standards, see Section 2.6.3). Two key resources with respect to the GCC aspect of the project will be the "Using the GNU Compiler Collection" (Stallman 2004*b*) and the "GNU Compiler Collection Internals" (Stallman 2004*a*) manuals. These are comprehensive references to both running and developing GCC and would be an appropriate resource for problems associated with GCC. Further documentation would be provided at a later stage as the project progresses.

Chapter 4

Design

4.1 System Architecture

As discussed in Section 2.2, the key phases of a compiler can be disassembled into high-level design topics. This will give an overview and understanding of the overall composition of the design. The basic components covered in the design will be the lexical analyser, symbol table, parser, generation of the GENERIC intermediate structures and the integration into GCC. The parser is central to the main design of this system, as it drives the front end and control the lexical analysis. It also builds the intermediate representations and passes control onto GCC when required.

4.2 Methodology

Expanding on the requirements analysis, the methodology for the design of the system was based upon a generic architectural model as defined in (Sommerville 2001). Generic models are a type of domain-specific architectural models which are abstractions from a number of real systems. They encapsulates the principle characteristics of these systems. A compiler model is a good fit as a generic architectural model and some of the important modules are described below. The phases of lexical, syntactic and semantic analysis would be organised sequentially as shown in Figure 4.1. The components which make up a compiler can also be organised according to different architectural models. A compiler could also be implemented using a composite model, where a data-flow architecture could be used and the symbol table acting as a repository for shared data. However, large systems rarely conform to a single architectural model, as they are heterogeneous and tend to incorporate different models at different levels of abstraction (Sommerville 2001). This means that this project will not be constrained by its adherence to the chosen model, rather it will be used as a guideline during the design and implementation stages.

4.3 Lexical Analyser

The lexical phase needs to provide the ability to "tokenise" the input language stream as desired. By using the generic model as described above, it would be feasible to create a lexical analyser module that takes the input language and converts it into recognisable symbols ((Richards & Whitby-Strevens 1980) points out that there are about 75 such symbols). This would ensure that the principle characteristics of the lexical phase are captured.

The lexical analyser would need to be driven by a starter application that processes the input and options from the compiler invocation. Minimal explicit setup would be required, as most of the internal data structures and rules are automatically created by Flex, but it would be sensible during development to enable certain diagnostic and debugging options (see (Paxson 1995) for further details). These options would then be removed in a production version of the compiler. The token types for the BCPL language would include identifiers, numeric constants, keywords and operators; each time one of these tokens were encountered, it would need to be successfully matched and handled appropriately. In many cases, it would be possible just to acknowledge and return that you have detected that particular token, but certain cases would require special handling and may require insertion into the symbol table. Other cases may require functionality that copies or adjusts the current token so that it can

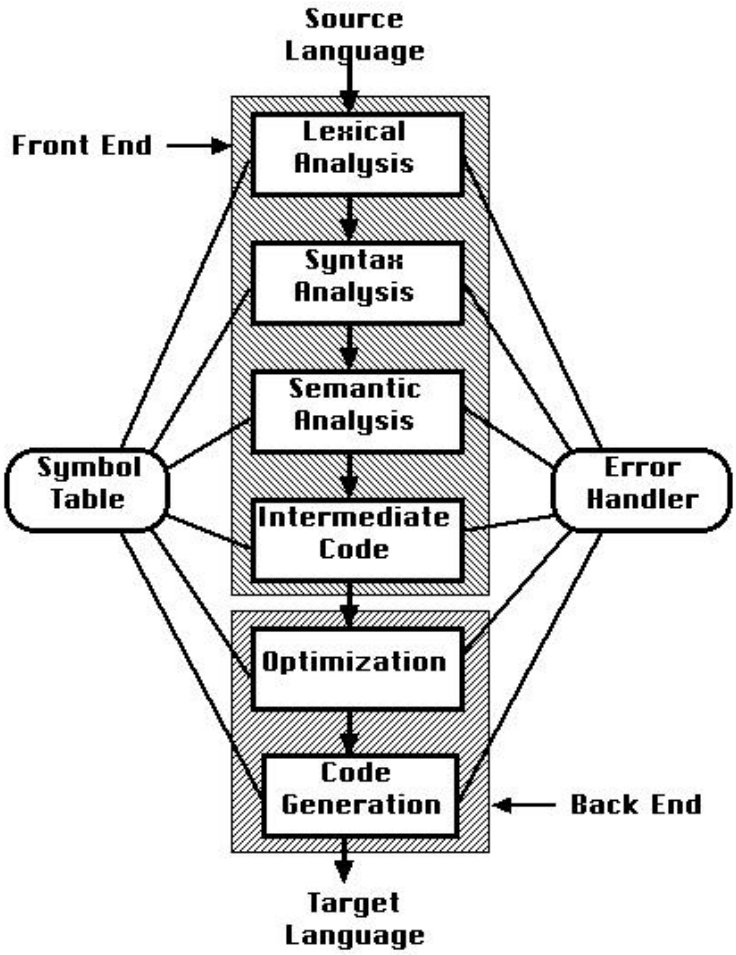


Figure 4.1: Compiler system overview

be passed onto the parser or be analysed further. Like the original BCPL compilers, the lexical analyser will not perform any backtracking; that is, it performs the analysis while reading the basic tokens in one at a time without having to consider a symbol previously dealt with (Tremblay & Sorenson 1985).

A common problem during lexical analysis is ensuring that you have correctly identified the current token, otherwise semantic mismatches can occur within the rules of the parser. A good way to alleviate this problem is to ensure that all keywords, commands and operators are explicitly defined as tokens in the declaration section of the Bison parser and are detected and returned as expected. It is also important to make sure that the alphabetical case of the input is taken into consideration, as some implementations of BCPL allowed both upper and lower cases to be used (Richards & Whitby-Strevens 1980), unlike Martin Richard's original compiler (Richards 1969). When using regular expressions in the rules to detect identifiers and constants for example, care must be taken to construct and test these expressions to check that the character classes are matching only the expected input. Nevertheless, the lexical phase is one of the few places where every character of input must be examined (obviously some more than others), so it makes sense to minimise the number of operations you perform per character (Aho et al. 1986). However, it is all too easy to make mistakes whilst using regular expressions in Flex for handling input streams (see common examples in (Paxson 1995)).

Another important design feature for the lexical analyser is error detection and recovery. It should be possible, whenever a error is detected in the scanner, to maintain control of the compiler without it crashing and also to receive information about the type, location and severity of the error. Certain errors or warnings may allow processing to continue, but some errors will require compilation to cease. The error information given back to the user should contain the error token, the line and column number of its location and the action problem. This would require tracking of whitespace and newlines, but should be trivial to implement. Giving as much error detail as possible is important from a diagnostic and debugging point of view and would also contribute to the testing of the program itself.

We have already covered the use of Flex in both Section 2.5.1 and Section 3.2.2, but there will be a more detailed technical discussion in the implementation of the lexical analyser later on in this report.

4.4 Symbol Table

The symbol table is the major persistent attribute of a compiler and, after the intermediate representations, forms the major data structure as well (Louden 1997). The efficiency of the symbol table in a large production compiler is an important issue, but the scope of this project precludes a large amount of time spent optimising symbol table transactions and storage size. However, the chosen design of using a separately-chained hash table for the symbol table representation is a good choice as it provides a compromise between efficiency and ease of implementation.

The efficiency of the symbol table depends on the implementation of its data storage (Louden 1997). Typical implementations include linear lists, various tree structures (such as binary search trees, AVL trees and B trees) and hash tables. Linear lists are a good basic data structure that are simple and quick to implement and can provide easy and direct implementation of the three basic operations. There exists a constant-time insert operation, by always inserting at the front or rear of the list, with lookup and delete operations being linear time in the size of the list (Loudon 1999). This option would be acceptable for a prototype or experimental compiler implementation, where speed is not a major concern. Search tree structures are somewhat less useful for the symbol table, partly because they do not provide best case efficiency, but also because of the complexity of the delete operation (Louden 1997). The hash table will often provide the best choice for implementing the symbol table, since all three operations can be performed in almost constant time and is used most frequently in practice (Aho et al. 1986).

A hash table consists of an array of entries, called buckets, indexed by an integer range. A hash function is required to turn the search key (usually the identifier name, consisting of a string of characters) into an integer hash value in the index range of the table, giving the location of the bucket where that item is then stored (Loudon 1999). Care must be taken to ensure that the hash function distributes the key indices as uniformly as possible over the index range, since hash collisions (where two keys are mapped to the same index by the hash function) can cause significant performance degradation in the lookup and delete operations. It is also important for the hash function to operate in constant time, or at least time that is linear to the size of the key (which would amount to constant time if there is an upper bound to the size of the key) (Loudon 1999).

The way we have chosen to address the problem of collision resolution in the hash table is using a technique called separate chaining, which is shown in Figure 4.2. In this method, each bucket is actually a linear list and collisions are resolved by inserting the new item into the front of the bucket list (Louden 1997). Another common method used is open addressing, where only enough space is allocated for a single item in each bucket, with collisions being resolved by inserting new items in successive buckets. This method was not chosen because of the performance degradation when collisions become frequent and also that deletions do not improve the subsequent table performance. The size of the initial bucket array is an important design choice that needs to be defined at compiler construction time. Typical sizes range from a few hundred to over a thousand buckets, but the use of dynamic allocation can ensure that small arrays will still allow the compilation of very large programs, at the cost of extra compile time. An interesting feature is that if the size of the bucket array is a prime number, a typical hashing function behaves more efficiently than with a composite number-sized array (Loudon 1999).

As mentioned in the requirements section, the symbol table will provide the principle functions of insertion, lookup and deletion. As the names suggest, insert is used to store symbols, lookup to retrieve them, and delete to remove them from external visibility. The delete function would not physically remove the entry from the table, but set a flag to make it inactive similar to a database record. This would provide a form of transaction history to ensure that items are not fully removed until the whole table is destroyed. These principle interfaces will be available to all phases that require access and be visible to the GCC middle and back ends during the lowering from GENERIC to GIMPLE form and the optimisation passes. The symbol table is intimately involved with both the scanner and the parser, either of which may need to enter information directly into the symbol table or query it to resolve ambiguities. A print table function will also be available that dumps the information in each bucket of

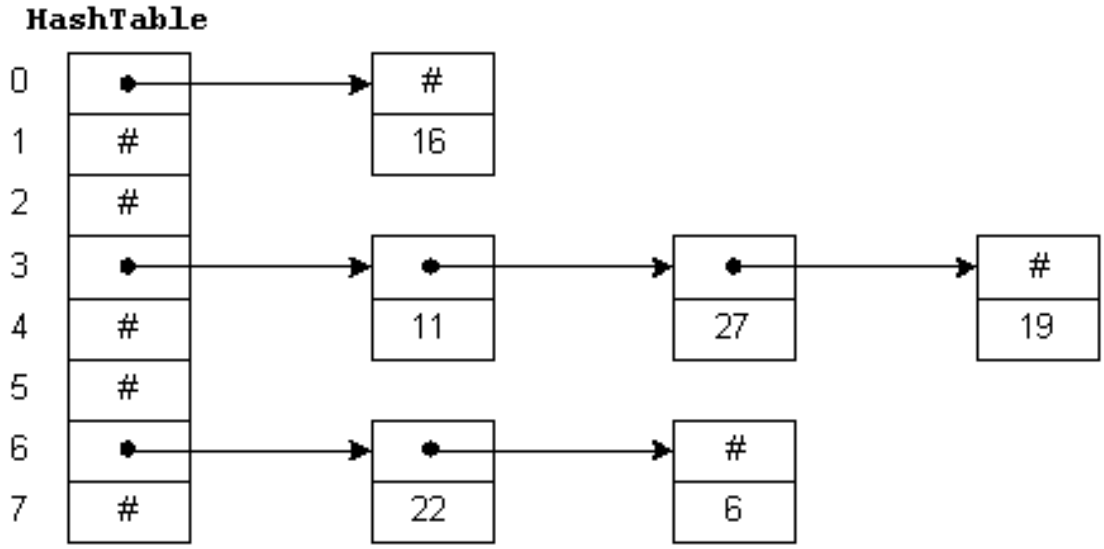


Figure 4.2: Separately-chained hash table composition

the symbol table to a file for diagnostic purposes. Other functions that would be available are a check to see if the table is empty, a symbol count function and a function to free the whole symbol table when the compile has ended.

As we have mentioned previously, BCPL is typeless in the modern sense, so the usual information about data types is not required for each entry in the symbol table. An important feature for BCPL is the block-structure of the language and how this constrains scoping of variables. A block in defined in BCPL as any construct that contains one or more declarations and commands (Emery 1986). BCPL permits the nesting of blocks inside other blocks and the scope of an identifier is the declaration itself (to allow recursive definitions), the subsequent declarations and the commands of the block. The scope does not include earlier declarations or extend outside the block (Richards & Whitby-Strevens 1980). Since block-structured languages permit multiple scopes for variables, this complicating the design of the symbol table as multiple scopes imply that a symbol may have different meanings in different parts of the code as highlighted in Figure 4.3. This example in pseudo-C demonstrates how it is possible to declare new variables at the start of every block and how this can complicate the program. This also highlights lexical scoping present in BCPL, C and most other languages descended from ALGOL. A variable is said to be lexically scoped if its scope is defined by the text of the program (Fischer & LeBlanc 1991). In this way, a programmer can guarantee that their private variables will not be accidentally accessed or altered by functions that they call.

4.5 Parser

The design of the parser is important in ensuring that the semantics and the grammar of the BCPL language are preserved. The parser is perhaps the most important module in the front end and drives the lexical phase by controlling the scanning and then working on the token that has been identified. It is still a separate module, but needs to be able to integrate cleanly with the scanner created in Flex. Again using the generic architectural model described earlier, it should be possible to develop a Bison parser that works on the data given to it by the lexical phase and then attempts to match it with the grammar productions. The principle characteristics of the system are to match the input against a set of grammar rules and to start building the intermediate representations with the information generated.

By using the Bison parser generator, we are able to take advantage of many automatically generated features. In order for Bison to be able to parse a language, it must be described by a context-free grammar. This means that you must specify one or more syntactic groupings and then give rules for constructing them from their constituent parts (Donnelly & Stallman 2002). For example, in BCPL, one such grouping is called an "expression", with one of the rules for making an expression being

```
1   static int w;              /* scope level 0 */
2   int x;
3   void example(a,b)
4   {
5       int a,b;               /* scope level 1 */
6       {
7           int c;
8           {
9               int b,z;       /* scope level 2a */
10              ...
11          }
12          {
13              int a,x;       /* scope level 2b */
14              ...
15              {
16                  int c,x;   /* scope level 3 */
17                  b = a + b + c + x;
18              }
19          }
20      }
21  }
```

Figure 4.3: Nested code blocks example

defined as "an expression can be made of a minus sign and another expression" (Richards & Whitby-Strevens 1980).

The most common formal system for presenting a grammar for humans to read is known as Backus-Naur Form or "BNF", which was originally developed in order to specify the language ALGOL 60. Any grammar expressed in BNF is a context-free grammar (see (Chomsky 1956) and (Chomsky 1959) for further details), though it may contain ambiguities. Therefore, the input to Bison could be thought of as a type of machine-readable BNF. This means that we are able to use the BNF grammar for BCPL as given in (Richards & Whitby-Strevens 1980) and build the necessary grammar productions. This is a good starting point to creating a working parser for BCPL, even though it may be necessary to make changes during implementation due to ambiguities and conflicts that may occur. This is a good point to note that whilst we are working from the BNF grammar as defined in (Richards & Whitby-Strevens 1980), no formal standard exists for the BCPL language. Attempts were made in the early 1980s to create a common standard (Middleton et al. 1982), but the large number of implementations combined with the waning usage of the language and the popularity of C contributed to it being unsuccessful. This means that the BNF grammar we have formalised is possibly the most comprehensive, but is does not include any of the commonly-used extensions or language features which are not expressible in the BNF notation. A further discussion about the changes made to the BNF grammar is described in Chapter 5, along with details of which extensions have been implemented.

The initial choice of the Bison parser implementation was tempered with research into developing a recursive-descent parser like the early BCPL compilers. Recursive-descent parsing is a top-down method of syntax analysis in which you execute a set of recursive procedures to process the input stream (Aho et al. 1986). This contrasts with using parser generators like Yacc or Bison, which produce bottom-up parsers. A recursive descent parser consists of a set of mutually-recursive procedures or non-recursive equivalents where each such procedure implements one of the production rules of the grammar. Thus the structure of the resulting program closely mirrors that of the grammar it recognises. Bison and most other parser generators produce a particular family of bottom-up parsers called LALR(1)[1] parsers. Look-Ahead LR (LALR) parsers are a specialised form of LR parsers that can deal with more context-free grammars than SLR parsers but less than LR(1) parsers can (Grune et al. 2000). It is a very popular type of parser because it gives a good trade-off between the number

[1]The notation used for describing parser types is $\{L|R\}\{L|R\}(k)$, where the first letter indicates which direction the input is read from, the second indicates the type of derivation produced (e.g. R indicates a rightmost derivation), while k refers to the number of unconsumed "look ahead" input symbols that are used in making parsing decisions. Usually k has a value of 1 and is often omitted.

of grammars it can deal with and the size of the parsing tables it requires. A detailed discussion about parser types and terminology can be found in (Aho et al. 1986).

This highlights some of the key features of using Bison, as the automatic generation of parsing tables and the internal manipulations of the stack to shift and reduce the productions considerably simplifies the task. It is also acts as a good starting point for understanding the language and its intricacies, as the tool provides a simple interface for building the parser. One of the problems with using a recursive-descent compiler is the greater amount of work involved for implementation, as it would have required a greater understanding of the language at the beginning of the project. It would probably in the long term have been a more elegant and efficient solution, but Bison was the choice for ease of implementation and the timescales involved. Also, with previous experience of using Bison, it is possible to reuse existing code for a C-like language rather than creating a parser from the beginning.

The design of the error recovery strategy is another important factor for the parser. This should be used in conjunction with simple error recovery in the lexical phase, as it is not usually acceptable to have the program terminate on the occurrence of the first parse error (Donnelly & Stallman 2002). For example, the compiler should recover sufficiently to parse the rest of the input file and continue checking it for errors. A common technique when a parse error is discovered is calling an error handling routine that advances the input stream to some point where parsing should recommence. It is then possible to provide error information for the whole input file, rather than stopping at the first error. However, error recovery strategies are by their very nature informed guesses; when an incorrect guess is made, one syntax error often leads to another (Donnelly & Stallman 2002).

4.6 Intermediate Representations

As previously mentioned, one of the key tasks of the parser is to build the GENERIC intermediate representation from the parsed input stream. It is hard to set design criteria on this part of the program, as we are constrained by the fact we are not using a custom intermediate representation. However, it is possible to state that when we build GENERIC trees, we will adhere to the recommendations set out in the GCC 3.5.0 version of the GCC Internals Manual (Stallman 2004*a*). This is also the case for the lowering pass to GIMPLE form, which is done by a function call provided by the GCC infrastructure.

4.7 GCC Integration

The GCC integration falls under the same situation as discussed above for the intermediate representations. As we are reusing the GCC middle and back ends for the optimisation passes and the code generation, we are unable to set or control any criteria of the design. Also, most of this stage of the compilation will be automatic once we have processed and passed control over to the GCC process. However, we will again adhere to the recommendations as set out in the GCC 3.5.0 version of the GCC Internals Manual (Stallman 2004*b*).

Chapter 5

Implementation

5.1 Overview

This chapter presents an overview of the software infrastructure implemented and a high-level discussion of the implementation process and the key components of the system. In this project, we have attempted to use an evolutionary and rapid prototyping strategy (Sommerville 2001) for implementation, as this seemed suitable for the type of project and the speed of development required.

5.2 Tools

The aim of this project has been to develop a BCPL compiler via integration with the proposed GCC 3.5.0 infrastructure and reuse the middle and back end. This design strategy heavily influences our choice of implementation tools, especially if we wish to maintain levels of interoperability, standardisation and support. The choice of tools has also been constrained by the need to support the design choices made in Chapter 4.

The choice of using C as the main implementation language was easy to make, as it has become the de facto language for compiler implementations. This is compounded by our use of GCC and the intermediate representations, as most of GCC is written in C. C++ is starting to be used for certain compiler implementations, as language features such as templates are useful when creating front ends (a good example would be the LLVM Project, as discussed in Section 2.5.6). A further deciding factor would be if we wished to eventually integrate the BCPL front end fully into GCC, the coding standards for both GNU (GNU 2003*a*) and GCC (GCC 2003*a*) would dictate our choice of language and tools. The expressive power of C, coupled with its common usage as a compiler tool are the primary implementation reasons. It is unlikely that choosing C as the implementation language would simplify certain features due to its relationship to BCPL.

5.3 Lexical Analyser

As discussed in Chapter 4, the purpose of the scanner is to recognise tokens from the input stream and match them against the corresponding tokens in the language. The choice of implementing the scanner in Flex was again easy to make, as Flex and its variants remain the standard tools for creating scanners for modern compilers. A Flex input file consists of three sections, *definitions*, *rules* and *user code* (Paxson 1995). An example set of rules from the scanner are shown in Figure 5.1. These demonstrate how basic tokens like keywords and operators are detected and their token type is passed to the parser. The scanner uses the enumerated token types as defined in the Bison parser.

However, for handling more complicated token such as identifiers and numeric constants, the use of regular expressions[1] were required to control the range of tokens captured. A good example from the definitions section of the scanner would be the regular expression for identifiers as shown in Figure 5.2.

[1] A regular expression (often abbreviated as regexp or regex) is a string that describes a whole set of strings, according to certain syntax rules. These expressions are used by many text editors and utilities to search bodies of text for certain patterns and, for example, replacing strings. The background of regular expressions lies in automata theory and formal language theory (Wikipedia 2004).

```
...
"GLOBAL"        { return GLOBAL; }
"LET"           { return LET; }
"WHILE"         { return WHILE; }
"TABLE"         { return TABLE; }
":="            { return ASSIGN; }
"!"             { return PLING; }
"->"            { return IMPLIES; }
...
```

Figure 5.1: Example Flex rules for a small subset of keywords and operators in BCPL

This shows how you can define character classes to create groups of characters to define larger entities. In the displayed example, an identifier consists of a string starting with a letter, followed by either an alphanumeric character, a period or an underscore.

```
...
letter          [A-Za-z]
integer         [0-9]
identifier      {letter}({letter}|{integer}|\.|_)*
...
```

Figure 5.2: Example set of Flex regular expressions for BCPL

The scanner performs tracking of section brackets (see Appendix C) by recording the entry and exit of a new block and hence new scope for the program. This means that when an opening bracket, `$(`, or closing bracket, `$)`, is found, a new scope is either opened or closed. A section bracket may be optionally tagged with an identifier to aid tracking of blocks; when this occurs the value associated with the opening bracket is pushed onto the stack. When a closing bracket is recognised by the scanner, the top of the stack is popped and compared with the tag and if they are the same the scope is closed. If they are different, the closing tag will be put back onto the input stream so that the next call to the scanner will attempt to match it against the new value at the top of the stack. The tracking of section brackets, and hence scope, is also important when adding and looking up entries in the symbol table. Therefore, an entity is only inserted into the symbol table if it is the first declaration or instance in that scope. If an entry already exists for that scope, a reference is made to the original entry.

It is possible to include a file in the source text of a program by using the get directive, which is comparable to the `#include` feature in C. A get directive is handled in the scanner by using a feature in Flex called exclusive start states that provides a mechanism for conditionally activating rules (Paxson 1995). A condition is declared in the definitions section of the scanner and then when the state is entered (when the `BEGIN` action is executed) only rules qualified with the exclusive start condition will be active. This enables the creation of "mini-scanners" within the main scanner and have been used extensively in this project. The other instances are for character constants, string constants and "eating" single-line and multi-line comments. By having certain rules for certain states, it is possible to scan portions of the input that are syntactically different from the rest (Paxson 1995). The benefit of using an exclusive state for the get directive is that you have more control over the switching of inputs and also you are able to control the depth of the nesting level.

Some of the commonly-used language extensions to BCPL have been implemented in the scanner, including:

- The option of using lower case letters, dots and underscores in identifiers and section bracket tags. This was done by adjusting the regular expressions governing identifiers and adding these to the character classes.
- The option of writing numeric constants in binary, octal or hexadecimal format. Again, this was done with changes to the character classes by creating new expressions for the new formats. A small change in the parser grammar was required to ensure the new numeric formats were recognised.

- The option of using `?` to represent an undefined constant, which may be different each time it is evaluated. Any constant expression containing an evaluated `?` is itself a `?`. This was implemented by allowing the scanner and parser grammar to recognise `?` as a valid constant, but without explicitly accepting a value.

- The full set of comment forms extend either from `//` or `||` or `\\` to end of line, or from `/*` or `|*` or `\*` to the corresponding close symbol (the two characters reversed). This is done by adding these formats to the exclusive states used for comments. Comments are not allowed to nest, and all comment symbols other than the required close symbol are ignored in a comment (Richards & Whitby-Strevens 1980).

Implementation of other language extensions have required small changes in the scanner or parser. Examples include the addition of an infix byte operator, floating-point operators (which are the same as their integer equivalents, prepended with `#`), field selectors (which allow quantities smaller than a whole word to be accessed with reasonable convenience and efficiency (Richards 1973)) and combined operator-assignments (the most recent implementations allowed the assignment symbol to be prefixed with any dyadic operator, for example `+:=`).

Whitespace and comments have been removed from the input stream, but both the line and column count are tracked so as to provide debugging location information on error. As discussed in the design, the scanner does not perform any backtracking for error recovery purposes. This technique can be time-consuming and is in some cases not particularly effective. The technique of accepting illegal input and then reporting it with an error or warning is a powerful one that can be used to improve the error reporting of the compiler (Levine, Mason & Brown 1995). Therefore, on finding an error, the line in which it occurs is skipped and the scanner resynchronises and continues scanning.

5.4 Symbol Table

As discussed in Chapter 4, the purpose of the symbol table is to store information about certain entities in the program throughout the entire compilation process. A discussion has already been made about the importance of efficiency in symbol table design, so the choice of using a separately-chained hash table is a good compromise between performance and speed of implementation. We have decided to reuse an existing symbol table, which has been taken from (Louden 1997) and adapted for our purpose. The main functions implemented are:

- An insert function to add a symbol, its scope and source code line number to the table.

- A lookup function to check if a symbol exists in the table; if so, the token is returned.

- A related boolean lookup function to check if a token exists in the table.

- A delete function to mark an entry as deleted (but not physically remove it from the symbol table.

- A size function to return the number of entries in the symbol table.

- A boolean function is check if the table is empty.

- A destroy function to free all allocations in the symbol table.

- A print table function to dump all symbol table entries to a file, listing their value, scope and line number.

The issue of scoping within the block structure was handled by inserting the scoping "depth" for each variable as tracked by the scanner. This means that variables are only added when they are the first instance in that particular scope. (Fischer & LeBlanc 1991) discusses the fact that in block-structured languages you do not usually have the scanner enter or lookup identifiers in the symbol table because it can be used in many contexts. It may not be possible, in general, for the scanner to know when an identifier should be entered into the symbol table for the current scope or when it should return a pointer to an instance from an earlier scope. However, it has been possible to resolve the scope of the variable by the tracking of the section brackets and also by following BCPL's lexical scoping rules (Richards & Whitby-Strevens 1980). The idea suggested of allowing individual semantic routines

to resolve the identifier's intended usage is not necessary.

The two important data structures which store the data in the hash table are `LineListRec` and `BucketListRec`, as shown in Figure 5.3. `LineListRec` is a list of line numbers of the source code in which the identifier is referenced and is stored as one of the fields in a `BucketListRec`. This is the record in the bucket lists for each identifier, and details the symbol name, scope and line list.

```
1           typedef struct LineListRec
2           {
3                   int lineno;
4                   struct LineListRec* next;
5           } *LineList;
6
7
8           typedef struct BucketListRec
9           {
10                  TOKEN* token;
11                  LineList lines;
12                  struct BucketListRec* next;
13          } *BucketList;
```

Figure 5.3: Symbol table data structures

A good hash function for the symbol table is of vital importance if efficiency and collision avoidance is required (Loudon 1999). The hash function converts a character string into an integer is the range zero to one less than the size of the table. Hashing can provide almost direct access to records, which means that, on average, a lookup can require just one or two probes into the symbol table to obtain the record. The hash function that we have used in our implementation is shown in Figure 5.4 and works through the character string to give an integer key by shifting modulo the table size. This is not the most complicated or efficient hashing function, but it is suitable for our purposes. As mentioned in the symbol table design, the table size has been set to a prime number (with the closest prime to 1024 being 1021) to potentially improve performance. Pre-loading of the symbol table with all of the keywords and commands was discussed during the design phase, but it did not seem to be worthwhile, especially as the performance benefits are debatable.

```
1   static int hash (char* key)
2   {
3           int temp = 0;
4           int i = 0;
5           while (key[i] != '\0')
6           {
7                   temp << SHIFT) + key[i]) % TABLE_SIZE;
8                   ++i;
9           }
10          return temp;
11  }
```

Figure 5.4: Symbol table hash function

5.5 Parser

The purpose of the parser is to analyse the tokens identified by the scanner in order to ascertain the semantic constructs in the input stream and build the relevant GENERIC nodes. In most compilers, the major portion of semantic analysis is to do with types inference and checking, which is not relevant for BCPL. Therefore, the role of the parser is important in ensuring the syntactic and semantic preservation of the language via the defined grammar. As we have chosen to build GENERIC nodes at each stages rather than via our own custom tree nodes, the process of parsing and the generation

of the intermediate representation are combined into a single phase. When a grammar production is matched, a relevant GENERIC node is built and when a top-level production is accepted, a GENERIC tree is returned.

The choice of using Bison to implement the parser was again trivial. Flex and Bison are designed to integrate and work together and still remain the standard tools for creating scanners and parsers. A Bison program consists of four main sections, *C declarations*, *Bison declarations*, *grammar rules* and *additional C code* (Donnelly & Stallman 2002). The important logic within the parser is contained within the grammar rules section, where production rules describe the grammar of the language in terms of terminals and nonterminals and how they decompose. An example set of production rules from the BCPL parser are shown in Figure 5.5, which descend from two arbitrary top level constructions called *program* and *program element* which are the entry points to the parser. This example demonstrates how declarations in BCPL decompose to four different types and would eventually decompose to a nonterminal symbol.

```
...
...
declaration             : simult_declaration
                        | manifest_declaration
                        | static_declaration
                        | global_declaration
                        ;

simult_declaration      : LET definition AND definition
                        | LET definition
                        ;
...
...
```

Figure 5.5: Bison production rules for declarations in BCPL

Recursion is another important property for developing production rules. A rule is recursive when a result nonterminal appears also on its right hand side (Donnelly & Stallman 2002). Nearly all Bison grammars need to use recursion, because it is the only way to define a sequence of any number of a particular entity. This is demonstrated in Figure 5.6, where manifest lists in BCPL can be composed of one or more manifest items. Unfortunately, many of the rules given in the original BNF grammar had no termination from their recursion (with examples being many of the expressions and lists). This required the addition of single drop-out nonterminals on the right hand side of the rule, as highlighted again by the manifest lists example.

```
...
...
manifest_list   : manifest_list SEMICOLON manifest_item
                | manifest_list manifest_item
                | manifest_item
                ;

manifest_item   : IDENTIFIER EQUALS const_expr
                ;
...
...
```

Figure 5.6: Bison production rules demonstrating left-recursion for manifest lists in BCPL

As previously declared, we are using the BNF grammar for BCPL as given in Appendix C, which has been adapted from (Richards & Whitby-Strevens 1980). On implementation, parts of the original version of the grammar were ambiguous (and hence could not be parsed by a LALR(1) parser) and did not represent all of the original language features. At first this was demonstrated by being unable to parse

simple code examples, but was later indicated by the large number of reduce/reduce and shift/reduce conflicts (see (Donnelly & Stallman 2002) for further information about the problems associated with these conflicts). By creating precedence and associativity rules within the Bison declarations section, it was possible to overcome these some of conflicts and use Bison's internal conflict resolution rules. However, as of 2004-04-05, the parser still reported 48 shift/reduce and 118 reduce/reduce conflicts. This is not an ideal situation, but at present it does not generate errors when tested with a wide range of code examples. Bison warns when there are conflicts in the grammar, but most real grammars have harmless shift/reduce conflicts which are resolved in a predictable way and can be difficult to eliminate (Donnelly & Stallman 2002). Coupled with the fact that Bison is designed to resolve shift/reduce conflicts by choosing to shift (unless otherwise directed by an operator precedence declaration), means that while the parser may require some attention it by no means indicates that the parser is broken. A good example of this would be before the GCC C++ parser was rewritten to use recursive descent for the 3.4.0 release, its Bison-generated parser reported over 30 shift/reduce conflicts and over 50 reduce/reduce conflicts.

A more significant problem with the original BCPL grammar is that not all of the original language features are represented in the BNF grammar, along with all of the commonly-used extensions. Some of the extensions implemented have already been described in Section 5.3, but certain features required changes to the parser. Some of the more important examples include:

- The most commonly-used keyword synonyms are for `THEN` and `DO` which are interchangeable in all circumstances, as are `OR` and `ELSE`. Also, `THEN` or `DO` may be omitted if immediately followed by another keyword or block (Willers & Mellor 1976). This was done by replicating the rules involving `THEN` and `DO` and checking that the proceeding token was "safe" and not one which would change the semantics of the statement from if the keyword was present. Appendix D gives a listing of some of the common operator synonyms.

- A particularly difficult language feature in BCPL allows the programmer in most cases to dispense with separating a command or line with a semicolon. This is an important point to note, because unlike C, the semicolon in BCPL is a separator rather than a terminator. The original BCPL compilers worked with the premise that if a semicolon is syntactically sensible at the end of a line, and one is not already there, it is inserted (Feather 2002). However, in order that the rule allowing the omission of most semicolons should work properly, a dyadic operator may not be the first symbol on a line (Richards 1973). This was implemented by an addition to the grammar allowing rules without semicolons for declaration parts and command lists. Unfortunately, this choice of implementation could possibly have been the cause for a number of reduce/reduce conflicts present in the parser. The reason for this is due to the fact that for two rules (depending on whether or not there is a semicolon between the declarations and the commands), the actions in each of these will be treated as different states even though they are the same. This leads to a reduce/reduce conflict, as the correct resolution cannot be known at the time the action must be performed because it depends on whether or not there is a semicolon between the declarations and commands. A similar situation occurs with the procedure definitions, as the action must be performed before it is known whether the definition is for a function or a routine (Page 2003). This would have have potentially been a trivial problem if it were a recursive descent parser, because it could have been possible to have simply set a flag when it expected a semicolon, to indicate to the scanner that it is possible to accept a newline instead of a semicolon.

- Certain constructions in BCPL can be used only in special contexts and not all of these restrictions are defined in the BNF grammar. The important examples of this problem are that `CASE` and `DEFAULT` can only be used in switches and `RESULTIS` only in expressions (Richards & Whitby-Strevens 1980). Another interesting example of this would be the necessity of explicitly declaring all identifiers in a program.

A syntactic ambiguity arises relating to the repeated command constructs shown in Figure 5.7. Command `C` is executed repeatedly until condition `E` becomes true or false as implied by the command. If the condition precedes the command (`WHILE` or `UNTIL`), the test will be made before each execution of `C`. If it follows the command (`REPEATWHILE` or `REPEATUNTIL`), the test will be made after each execution of `C`. In the case of line 3 in Figure 5.7, there is no condition and termination must be by a transfer or `RESULTIS` command in `C`, which will usually be in a compound command or block. The resolution comes around by declaring that within `REPEAT`, `REPEATWHILE` and `REPEATUNTIL`, `C` is taken as short as possible. Thus, for example `IF E THEN C REPEAT` is the same as

`IF E THEN $( C REPEAT $)` and `E := VALOF C REPEAT` is the same as `E := VALOF $( C REPEAT $)`.

```
1           WHILE E DO C
2           UNTIL E DO C
3           C REPEAT
4           C REPEATWHILE E
5           C REPEATUNTIL E
```

Figure 5.7: Syntactic ambiguities in BCPL repetitive commands

When an error is detected during parsing, Bison is left in an ambiguous position (Donnelly & Stallman 2002). Therefore, it is unlikely that meaningful processing can continue without some adjustment to the parser stack. There is no reason why error recovery is necessary, but it may be possible to improve productivity for the programmer by recovering from the initial error and continue examining the file for additional errors. This technique shortens the edit-compile-test cycle, since several errors can be repaired in each iteration of the cycle (Levine et al. 1995). By using the `error` token provided by Bison for managing recovery, we can attempt to find a synchronisation point in the grammar from which it is likely that processing can continue. Unfortunately, the emphasis is on *likely*, as our attempts at recovery may not remove enough of the erroneous state to continue and the error states may cascade. At this stage, the parser will reach a point where processing can continue or it will abort. The use of the `error` token means that after reporting a syntax error, Bison discards any partially parsed rules until it finds one in which it can shift the `error` token. It then performs resynchronisation by reading and discarding tokens until it finds one which can follow the `error` token in the grammar (Levine et al. 1995). A key point, therefore, is the placement of error tokens in the grammar. The two conflicting goals are between placing it at the highest level possible, so that there will always be a rule to which the parser can recover, against the fact you want to discard as little input as possible before recovering by minimising the number of partially matched rules the parser has to discard during recovery (Donnelly & Stallman 2002). It was decided to place the `error` token at the highest level possible and resynchronise after the next newline, so as to aid maximum recovery during the development stages of the compiler. This method of ad-hoc recovery provides the opportunity to anticipate the possible occurrences of errors and in particular their position. A more detailed discussion of ad hoc and syntax directed error recovery techniques can be found in (Tremblay & Sorenson 1985).

The most difficult task of the parser is to start to build the intermediate representations to pass over to the GCC infrastructure. This part of the development is where the most technical knowledge is required, as the use of Flex and Bison has been supported by the long life of these tools and the large amount of resources available. However, the elegance of re-sourcing GCC is matched by its immediate difficulty with both the building of the intermediate representations and the integration into the main GCC framework. The possible implementation of the intermediate representations is discussed in detail in the following section.

5.6 Intermediate Representations

5.6.1 Overview

The project code was forked into two branches at this stage, as it was deemed important to provide a contingency plan if problems occurred in the implementation and integration of the GCC infrastructure. Therefore, the code was frozen at a stage where the scanner and parser could parse a wide range of BCPL test code. The side branch consisted of the GENERIC node building and a rudimentary BCPL system library, along with some general improvements to the compiler infrastructure.

Our main resource of existing code is from the C and Fortran front ends[2] taken from the GCC 3.5.0 daily snapshot (previously `tree-ssa` branch, available from (GCC 2004*b*)). The C front end unfortunately uses a combination of GENERIC and GENERIC trees in the same routines (see Section 2.4.2),

[2]Details about the format of the GCC source directory can be found at `http://gcc.gnu.org/onlinedocs/gcc-3.4.0/gccint/Top-Level.html`, but most of the main GCC source is contained with the *gcc* subdirectory. The format of the GCC subdirectory can be found at `http://gcc.gnu.org/onlinedocs/gcc-3.4.0/gccint/Subdirectories.html`.

plus a handful of language specific tree codes defined in `c-common.def`. We have also investigated the `gimplify_expr` function, which either returns GIMPLE directly or a GENERIC version of the given node to be handled by the `gimplifier`. This seemingly poor modularity has arisen as an artifact of the way the infrastructure evolved, as the C front end was the driver for the development of the `tree-ssa` branch. The Fortran front is probably a purer implementation of the infrastructure, but because of the relationship between BCPL and C, it makes sense to study and reuse the code for similar language constructs. Therefore, when presented with a C or C++ source program, GCC parses the program, performs semantic analysis (including the generation of error messages), and then produces the intermediate representation. This representation contains a complete representation for the entire translation unit provided as input to the front end. It is then typically processed by the code generator in order to produce machine code (Stallman 2004*a*). A more detailed discussion about the integration of the front end to the rest of the GCC infrastructure in described in Section 5.7.

The main purpose of GENERIC has been to provide a language-independent way of representing entire functions in trees (Stallman 2004*a*). As mentioned earlier, a problem with using GENERIC is the apparent lack of documentation and the fact that many parts of it are still under active development. An unfortunate quote that seems to sum up the situation is if you are able to express it with the codes in the source definition file `gcc/tree.def`, it is in GENERIC form. It is not a ideal situation to be writing code in a language that is only defined in its own code implementation. The existing front end "How-To" documents have yet to be updated to reflect the significant changes in the intermediate representation infrastructure and as such are not relevant. GIMPLE is a simplified subset of GENERIC for use in optimisations and was heavily influenced by the SIMPLE intermediate language used by the McCAT compiler project at McGill University (Hendren et al. 1992). In GIMPLE, expressions are broken down into three-address form, using temporary variables to hold intermediate values for use in the later optimisation phases. Also, control structures are lowered to `goto`s (Stallman 2004*a*).

5.6.2 Trees

The central data structure used by the internal representation is the tree (Stallman 2004*a*). The tree nodes, while all of the C type `tree`, are of many varieties, as they can be pointers to many different types. The `TREE_CODE` macro is used to tell what kind of node a particular tree represents. The front end needs to pass all function definitions and top level declarations off to the middle end so that they can be compiled and emitted to the object file. This means that this is done as each top level declaration or definition is seen, as GENERIC represents *entire* functions as trees. The BCPL front end at present only builds rudimentary GENERIC trees for certain constructs in the grammar and attempts to attack some of the more difficult language features by working from the C and Fortran front ends. It may be necessary to create some language-dependent tree codes for our implementation, as BCPL language features such as `RESULTIS` and `SWITCHON` have caused problems during implementation. Some library functions, such as `LONGJUMP` and `LEVEL` may also require special handling due to problems of representation in GENERIC form. This is an acceptable approach, so long as these custom codes provide hooks for converting themselves to GIMPLE and are not expected to work with any optimisers that run before the GIMPLE conversion pass. Currently, all optimisation passes are performed on the GIMPLE form, but it would be possible for some local optimisations to work on the GENERIC form of a function; indeed, the adapted tree inliner works fine on GENERIC, but the existing GCC compiler performs inlining after lowering to GIMPLE (Stallman 2004*a*).

As stated previously, we have only been able to implement a small subset of the functionality of BCPL in GENERIC tree form. This has been partly due to the steep learning curve of working with GENERIC, but also the problems of actually validating the GENERIC nodes produced. Therefore, the following set of implementation choices have been made concerning particular BCPL language features. This is not an exhaustive list of either the intended implementation, or of the features available in GENERIC. A detailed description of the important nodes and constructs in GENERIC can be found in (Stallman 2004*a*).

5.6.3 Identifiers

Identifiers have been implemented in GENERIC by using the `IDENTIFIER_NODE`. This construct does represents a slightly more general concept than the standard C or C++ concept of an identifier, which is turn is slightly different in BCPL, as an `IDENTIFIER_NODE` may contain a `$` or other extraordinary characters (Stallman 2004*a*). An important point is that there are never two distinct `IDENTIFIER_NODE`s

representing the same identifier, so it is possible to use pointer equality rather than using a routine like `strcomp` to compare `IDENTIFIER_NODEs`. Two important macros that are available whilst working with `IDENTIFIER_NODEs` are the `IDENTIFIER_POINTER` which is the `NUL`-terminated string represented by the identifier as a `char*`; and the `IDENTIFIER_LENGTH`, which is the length of the string, not including the trailing `NUL`, returned by `IDENTIFIER_POINTER`. Therefore, the value of `IDENTIFIER_LENGTH(x)` is always the same as `strlen(IDENTIFIER_POINTER(x))` (GCC 2003*d*).

5.6.4 Declarations

There are two basic kinds of declarations in BCPL, section declarations and simultaneous declarations. Identifiers declared in section declarations have a scope that starts at the end of that identifier's declaration; identifiers declared in simultaneous declarations have a scope that starts at the first of the set of simultaneous declarations (Richards & Whitby-Strevens 1980). In each case, the scope ends at the end of the block the declaration occurs in, or the end of the file if not in a block (Feather 2002). All declarations except function declarations are implemented similarly in GENERIC. The macro `DECL_NAME` returns an `IDENTIFIER_NODE` giving the name of the entity stored. Most declarations in BCPL can be easily handled by the various kinds of declaration nodes available, such as `LABEL_DECL` to define labels such as label prefixes, `CASE` labels and `DEFAULT` labels. It may be possible to use the `RESULT_DECL` node to implement `RESULTIS`, which is used with `VALOF` blocks, by converting it to a function declaration and then this could be used as the return value for the `VALOF` block. However, this is currently unimplemented, but remains a potential approach for implementation.

There exists scoping issues with declarations in BCPL, especially with rules for certain language constructs. For example, labels have the form of an identifier followed by a colon and any number of labels may precede a command. A label is only in scope within the smallest of:

- The commands (but not declarations) of the textually surrounding block.
- The body of the textually surrounding routine (if any).
- The body of the textually surrounding `VALOF` (if any).
- The body of the textually surrounding `FOR` (if any)

These rules may be difficult to implement within the GENERIC framework, but it may be possible to use the predicates that control the level of scoping, such as `DECL_CLASS_SCOPE_P`, which holds if the entity was declared at a class scope; or the predicate `DECL_FUNCTION_SCOPE_P`, which holds if the entity was declared inside a function body (Stallman 2004*a*). Nevertheless, this would require more design and development time.

A valuable feature of BCPL is the existence of manifest constants which give the ability to use names to represent constants. The value associated with a manifest constant stays fixed, so you cannot assign to it (Richards & Whitby-Strevens 1980). This is similar to the C preprocessor directive `#define`, except they are restricted to values which can be stored in a single cell and the scope of a manifest constant is the same as that of a identifier declared in the same position. The problem with the implementation of manifest constants is the fact that they can be used in constant expressions, which are required at compile time rather than runtime. Further research is required to see if functionality exists to provide a binder that could encapsulate the manifest constant and then could be replaced by its value wherever it is used.

5.6.5 Functions

Both functions and routines exist in BCPL, with arguments passed strictly by value (Richards & Whitby-Strevens 1980). The corresponding difference between the two is that a functional application is an expression and yields a result, whereas as a routine call is a command and does not. In common practice, the word procedure is used to mean either in contexts where the distinction is unimportant (Richards 1967). However, in GENERIC, they are both represented as functions by using the `FUNCTION_DECL` node. Arguments are accessed using the `DECL_ARGUMENTS` node, which returns the `PARM_DECL` for the first argument to the function (GCC 2003*d*). As in C, nested functions in BCPL can be handled by using the `DECL_CONTEXT` macro, which indicates that the context of a function is another higher level function and that the GNU nested function extension is in use. One problem with this is

that nested functions can refer to local variables in its containing function; in certain contexts, this may potentially violate some of the BCPL lexical scoping rules. In BCPL it is legal to nest procedures, but all variable references must be fully global or fully local: inside an inner procedure, variables local to the outer procedure are out of scope. Thus in comparison to C, you get some additional name hiding, but the compiler does not have to maintain static links (Feather 2002). Some further investigation is required to ensure that the scoping semantics are robust for the choice of implementation, but the range of constructs and predicates available in GENERIC should enable an appropriate solution. For example, `DECL_LOCAL_FUNCTION_P` can be used if the function was declared at block scope, even if it has global scope (Stallman 2004*a*).

5.6.6 Statements

There exists corresponding tree nodes for all of the source-level statement constructs used within the C and C++ front end. This means that most, if not all, of the BCPL statements should fit into one of these nodes. In GENERIC, a statement is defined as any expression whose value, if any, is ignored (Stallman 2004*a*). A statement will always have a `TREE_SIDE_EFFECTS` set (or it will be discarded), but a non-statement expression may also have side effects. Many of the statements will have sub-statements when they are represented in a GENERIC node. For example, a BCPL `WHILE` loop will have a body, which is itself a statement. If the sub-statement is `NULL_TREE`, it is considered equivalent to a statement containing a single `;` i.e. an expression statement in which the expression has been omitted. Other implementation examples are `DECL_STMT` for local declarations, possibly using a GCC extension allowing declaration of labels with scope. Also `DO_STMT`, `FOR_STMT`, `GOTO_STMT`, `IF_STMT`, `RETURN_STMT`, `SWITCH_STMT` and `WHILE_STMT` to represent some of the common unlabelled commands in BCPL. Scoping is handled by using the `SCOPE_STMT` node; therefore, when a new scope opens (for example when entering a new block), the predicate `SCOPE_BEGIN_P`, will be set and at the end of a scope, the predicate `SCOPE_END_P` will hold (GCC 2003*d*).

5.6.7 Expressions

The GENERIC representations for expressions are for the most part quite straightforward and nodes exist for many of the common expressions in BCPL. However, it is important to consider the fact that the expression "tree" is actually a directed acyclic graph (DAG). This means that there may be numerous references to a node throughout a source program, many of which will be represented by the same expression node. Therefore, you cannot rely on certain kinds of node being shared or being unshared. The macro `TREE_TYPE` is used to return the type of the expression stored in the node (Stallman 2004*a*). The range of GENERIC nodes are far too numerous to list here, but again most, if not all BCPL expressions are covered. For example, conditional expressions are handled by `COND_EXPR` and function calls by `CALL_EXPR`. String constants are handled by the `STRING_CST` node, but it is important to consider the BCPL string implementation. They are word packed and not `NUL`-terminated, with the first byte storing the length of the string. A function for converting between BCPL and C strings has been implemented in this project. In some implementations that did not have an infix byte operator, the only way to reference individual characters was to shift and mask (Feather 2002).

BCPL's logical and relational operators have direct equivalents in C and can be easily represented as GENERIC nodes. However, the clear distinction in C between the bitwise and boolean forms is not present in BCPL, as the distinction depends on context (Richards 1973). In C, `true` can be represented by an non-zero value, whereas it is useful in BCPL for the value representing `true` to be the bitwise negation of the value representing `false`. An example of this would be when a BCPL program assigns a value from a logic expression to a variable that is then used as the test in an `IF` statement, the test will have the desired result. It is worth noting that this behaviour is not actually defined by the language definition and the result can be implementation dependent (Richards & Whitby-Strevens 1980). Therefore, in BCPL, logical operations are evaluated according to one of two rules. If the result of the operator is in *truth context*, then the truth rules are used. Otherwise the operator takes bit-patterns as arguments and operates on each bit separately. Truth context is defined as:

- left hand side of $\rightarrow$ (implication)
- outermost operator in the controlling expression of a conditional command
- a direct operand of an operator evaluated using truth rules.

This situation also requires further investigation, as it is imperative that the implementation of the logical operators is consistent and semantically correct. A trivial implemented example is of bitwise equivalence (`EQV`) in BCPL, which can be easily represented as the negation of an XOR node, `BIT_XOR_EXPR`.

5.6.8 Global Vector

The global vector, where system and user variables are stored in fixed numerical locations, is the sole means of communication between separately compiled segments of program (Richards & Whitby-Strevens 1980). The first 150 locations are usually allocated to the operating system for many of the functions in the system library, but the declaration `GLOBAL $( N1:K1 $)` associates the identifier `N1` with the location `K1` in the global vector.

There are two major challenges in dealing with the implementation of the global vector. Firstly, it is allowed to be arbitrarily large, so its size is not known until all of the modules in a program have been compiled. Secondly, that a global declaration results in an identifier referring to the cell within the global vector and not a variable containing a pointer to a cell, though this can be dealt with in a similar way to manifest constants. The proposed implementation is to create a C library wrapper that declares an `extern` variable to point to the global vector in the BCPL "world". This then gives us an opportunity to preallocate the essential operating system library functions into the global vector. Also, by having a variable that points to the base of the global vector throughout compilation, we know that once it has finished and the size of the global vector is known (as all modules would have been compiled and linked), it would be possible to generate the code to create the global vector and start the program.

5.6.9 System Library

Most BCPL implementations comprise of a set of basic procedures, together with a standard library written in BCPL. These basic procedures provide a means of accessing functionality within the operating system and machine-level facilities such as input and output (Richards & Whitby-Strevens 1980). The system library has been built up by consensus over the years, so it is not always possible to guarantee a particular procedure always being in a particular place in the global vector. All system routines and functions are accessed via the global vector which is populated by the global declarations in the system header `LIBHDR`. The allocation of routines and functions to particular cells is implementation defined, but only cells less than `FIRSTFREEGLOBAL` (a manifest constant) are used (Feather 2002). The chosen method of implementation is obviously related to how the global vector is implemented, as that is the container for all of the procedures. Since it is possible to access the externally declared global vector, we can simply pre-populate it with our own implementations of the most important functions. This means that the C library wrapper essentially re-implements base functions such as `RDCH` (input) and `WRCH` (output) as calls to the C I/O library. All we need to do is pass a pointer to these calls into the global vector and then it will be possible to access this functionality from within the BCPL program. Examples of this would be setting `CIS` (Current Input Stream, usually global position 24) and `COS` (Current Output Stream, usually global position 25) to `stdin` and `stdout` respectively.

The two most important functions that have been implemented as C system calls are for input and output. `RDCH`, which returns the next character from the selected input stream, and `WRCH`, which writes the character to the selected output stream. These are handled by declaring functions that call the C library functions `getc` and `putc`. Pointers to these functions are then passed to the relevant position in the global vector and then can be called as normal. This means that input and output functions such as `READN` and `WRITEN` should work, as these just make calls to `RDCH` and `WRCH` respectively. Other important implementations have been for the entry and exits points to a BCPL program, `START` and `FINISH`.

While it remains to be seen how well this will work when implemented in GENERIC, the choice of implementation is sufficiently robust enough to cope with getting a subset of BCPL library function working to test simple functionality. Other possible implementations have been to physically rewrite the whole BCPL library in C, but this was deemed to be too time-consuming even if it would probably be more efficient. The idea of attempting to look at the BCPL application binary interface and see if it would be possible to link in the existing library at compile time are not unrealistic, but would require a large amount of investigation into how the BCPL binaries are constructed and how they could be linked.

5.6.10 Miscellaneous

This section describes miscellaneous language features and how they have been implemented in the intermediate representations.

It is impossible to discuss developing a BCPL compiler without tackling the problem of conversion from word addressing to byte addressing. The use of word-length data objects in BCPL (called *cells*) causes problems on byte-addressed machines, as BCPL defines consecutive words in memory to have numerically consecutive addresses (Richards & Whitby-Strevens 1980). Hence, given a pointer to one such word, `p`, you can access the pointer to the next words by writing `p+1`. This is the same as for a `VECTOR` object in BCPL, as this is represented by a variable holding a pointer to its first word. So if `v` is a `VECTOR` then the value `v` points to the first word, `v+1` to the next, and `v+i` to the `i`th. The dereferencing operator (known as "pling") in BCPL is written `!`, so `!(v+i)` gets you the `i`th component of the `VECTOR v`. This can also be written as `v!i`, using the dyadic form of `!`. Recall, however, that BCPL is typeless. The compiler would not know that `v` is a vector and `i` an index; they are both just bit patterns. Hence, the expression `v+i` looks just like normal integer addition. This has a strange consequence: if `v+i` is the same code, regardless of whether `v` is an integer or a vector, then pointers in BCPL to adjacent words must indeed differ by 1 (Emery 1986). This means that on a machine with an underlying byte-addressing scheme, BCPL pointers cannot be true addresses, but must be scaled by 2 (for example on the Intel x86 or PDP-11) or by 4 (on the Motorola 68000) before being dereferenced. If memory is byte-addressed rather than word-addressed, features in BCPL become less attractive. The compiler either has to painfully construct byte addresses, or you drop the requirement that adding 1 to an address gives you the address of the next word. The first is inefficient for the operations `lv` and `rv` (like unary `&` and `*` in C), but becomes a two-way choice for real array access: to evaluate `E1[E2]` in C you would normally scale `E2` by the size of the object pointed to by `E1` (or vice versa, as it can be shown that `E1[E2]` is equivalent to `E2[E1]` (Richards & Whitby-Strevens 1980)), whereas in BCPL you would just add `E1` to `E1` and scale the sum). The second approach causes most existing BCPL programs to break. Therefore, indirection and addressing referencing is a problem, but as long as references are consistently shifted before calculations are made, then it should be possible to use the GENERIC `ADDR_EXPR` and `INDIRECT_REF` nodes. Again, these features require more work and testing before full functionality exists.

An interesting language feature in BCPL is the `TABLE` operator, which gives an initialised, permanently allocated vector (Richards & Whitby-Strevens 1980). It is also one of the few operators in BCPL that associates right to left (see Appendix D for a table listing operator associativity). The expression `TABLE k1,k2,...kn` returns the address of `n` contiguous cells initialised to `ki` in order. Since all the values are constant expressions, they must all be evaluated at compile time. A possible way of implementing this could potentially involve using the GNU C extension `({ ... })`, which allows several statements between the two brackets and yields an expressions which is the value of the last one (ensuring you take into account the shift required to convert to word address on 32-bit machines). There does not seem to be a simple way of implementing it in ANSI C (Manning 1999). It may be possible to implement it in GENERIC as a function declaration which returns the appropriate value, but this raises problems as we are in an expression, so declarations are not allowed, as the function must be declared before the current function (Richards & Whitby-Strevens 1980). Also, the values, though constant, could still depend on manifest constants which are in scope only in the current function. The BCPL `VALOF` expression also encounters similar problems. The expression `VALOF c`, where `c` is a command, causes `c` to be executed. If, during the execution of `c`, a `RESULTIS` command is reached, that controls the value of the expression, otherwise the value is unspecified. Therefore, it may be possible to implement the `VALOF` construction as a type of function declaration, but more research is required for both this and the implementation of the `TABLE` operator.

The infrastructure available for handling errors in the intermediate representation is done by using the special `error_mark_node`, which has the tree code `ERROR_MARK` (Stallman 2004*a*). If an error has occurred during front end processing, the flag `errorcount` is set. If the front end encounters code that it can not handle, it will issue a message to the user and set the flag `sorrycount`. When these flags are set, any macro or function which normally returns a tree of a particular kind may instead return the `error_mark_node`. At present, no processing of erroneous code is implemented, but this would be possible by using and handling the `error_mark_node`.

5.7 GCC Integration

The implementation of this part of the project has not advanced beyond a proposed implementation stage, due to problems discussed in the previous sections. However, it is possible to describe some of the important language features and potential problems that need to be addressed.

The existing GCC infrastructure (up to GCC 3.4.0) is shown by Figure E.1 in Appendix E. This shows the how each language front end goes from the language-specific tree representation to RTL, before being passed to the back end for the optimisation passes and code generation. However, as previously described in Section 2.3 and Section 2.4.2, the new framework for GCC 3.5.0 has some significant infrastructure changes. This new structure is shown by Figure E.2 in Appendix E. The three main intermediate languages that GCC uses to represent the program during compilation are GENERIC, GIMPLE and RTL (Stallman 2004*a*). GENERIC serves as the interface between the parser and the optimiser, while GIMPLE and RTL are used during program optimisation. Most of the work of the compiler is done on RTL, with the instructions to be output described in an algebraic form describing the action. GIMPLE is used for target and language independent optimisations, so we do not need to worry about the lowering pass from GENERIC to GIMPLE, as this functionality exists in the infrastructure (`gimplify_function_tree`). Our only difficulty may occur if we decide to implement special language-specific tree nodes, as we would need to provide our own method of lowering to GIMPLE. Therefore, a large discussion of GIMPLE is not necessary, but further details about its nodes and format can be found in (Stallman 2004*a*), while a rough GIMPLE grammar can be found in (Merrill 2003). This therefore means that once the GENERIC representation is built and lowered to GIMPLE form, it can be passed onto the GCC infrastructure and automatically processed. The only actions that would require attention would be the handling of errors, but if the GENERIC form had been built correctly and lowered without any problems, they would be errors outside of the project scope and control.

Most of the tree-based optimisers rely on the data flow information provided by the Static Single Assignment (SSA) form. GCC implements the SSA form as described in (Cytron et al. 1991), which is produced from a conversion of the GIMPLE form. The compiler modifies the program representation so that every time a variable is assigned in the code, a new version of the variable is created. The process is controlled by the source code in `tree-ssa.c` (Merrill 2003). Although this process would be automatically controlled via the GCC back end, it is important to understand how the optimisation passes are implemented and the range of optimisations that occur. The SSA form depends on the control flow graph (CFG) data structure that abstracts the control flow behaviour of the function that is being compiled and is built on top of the intermediate code (the RTL or tree instruction stream). The CFG is a directed graph where the vertices represent basic blocks and the edges represent possible transfer of control flow from one basic block to another (Stallman 2004*a*). It is important to ensure that each compiler phase keeps the control flow graph and all profile information up-to-date, as the CFG also contributes to maintaining liveness information for registers. Further details about the SSA form and the implemented optimisations can be found in (Stallman 2004*a*).

Another key feature that has yet to be implemented is the problem of providing an interface for the intermediate representations to access the symbol table. Unfortunately, this is not documented in the GCC Internals Manual (Stallman 2004*a*), but a possible implementation may be to build and attach `BLOCK` nodes, containing declaration chains, to `BIND_EXPR` nodes. This is the technique that is used in the GNU Fortran (`gfortran`) front end (GNU 2004*a*) and may be possible to adapt for our usage.

The issues with the lack of documentation and resources about GCC integration and building GENERIC trees has been a major problem with this stage of the development. The only current and authoritative resources have been the GCC Internals Manual (Stallman 2004*a*) and the Tree SSA online documentation (GCC 2003*d*), which have been used extensively. However, the GCC Internals Manual is currently undergoing major rewrites to document many of the features discussed and implemented in this project. For many of the critical features that required further information, the documentation has been either incomplete or missing (frustratingly, there are numerous instances of "documentation incomplete" in certain sections of the manual). This contributed to a very slow development progress for building the intermediate representations and very little progress with the integration into GCC. However, a helpful section in the GCC Internals Manual has been the "Anatomy of a Front End" (Stallman 2004*a*), which describes in some detail the configuration files expected by GCC for building and using a new language front end, which has been used extensively. As described earlier, the proper way to interface GCC to a new language front end is with the `tree` data structure, which is described in the source files `tree.c`

and `tree.def`. Unfortunately, there remains numerous sections which are incomplete and the manual freely admits it is only preliminary documentation.

Chapter 6

System Testing

While the latter parts of this dissertation has focused on developing a new front end for GCC, a key requirement has always been ensuring that the semantics of the BCPL language are preserved. This has been a cause of many of the encountered implementation problems, from scoping issues to word addressing. Because of the problems encountered whilst building the intermediate representations and with the integration of GCC, the lack of an existing framework to support exhaustive testing means that the main focus has been on testing the scanner, parser and symbol table. The testing of the working system focuses on determining if the system meets the requirements as specified in Chapter 3. The results for a selection of the tests can be found in Appendix F, though a short overview of the testing process is discussed further in the following section.

Verification and validation are the names given to the checking and analysis processes that ensure that software conforms to its specifications (Sommerville 2001). Verification and validation are often confused but they are not the same. A succinct definition that highlights the differences between the two is that validation can be thought of as "are we building the right product?" and verification as "are we building the product right?". The two common techniques that we have used for our basic testing strategy are software inspections and software testing. Software inspections are static techniques that involve analysing and checking system representations such as the requirements document, design diagrams and the source code. Software testing involves executing the software with test data and examining the outputs of the software and its operational behaviour to check that it is performing as desired (Sommerville 2001). Both of these are simple, commonly-used techniques and were deemed appropriate for the type and stage of development. It did not seem particularly worthwhile to spend a large amount of time creating and performing a complex testing strategy when the testing performed throughout development was sufficient. At the time of writing, the compiler was still in a prototype stage and has been developed to explore functionality and for evaluation purposes. Since it has no security or safety critical application, exhaustive testing is not necessarily appropriate and has not been one of the key deliverables of this project. Nevertheless, in the future, it would be sensible to implement some form of formal testing strategy, possibly involving black box testing (see (Sommerville 2001) for further information).

Therefore, the testing and debugging cycles that have been performed during the development have been sufficient to catch a significant amount of errors. This has been helped by the use of diagnostic option flags for Flex, Bison and GCC, which have contributed to better debugging in both the scanner and the parser, plus stricter checking of the C code. For example, GCC is invoked with flags to enforce and issue all the warnings demanded by strict ISO C, along with others that report constructions which are not inherently erroneous but which are risky or suggest there may have been an error (Stallman 2004*b*). It is also invoked with the option to produce debugging information for use by GDB (GNU Debugger) or DDD (GNU Data Display Debugger), which has been used for testing, along with Splint, which is a tool similar to lint for statically checking C programs for security vulnerabilities and coding mistakes. Flex is invoked in debug mode, so that whenever a pattern is recognised the scanner will output information about the matched rule and the line it occurred on. Messages are also generated when the scanner backs up, accepts the default rule or reaches an end-of-file (Paxson 1995). Bison is also invoked with verbose debugging options set, which creates an extra output file containing detailed descriptions of the grammar and parser (Donnelly & Stallman 2002). This was extremely useful when attempting to find the cause of conflicts and grammar errors, in particular when attempting to resolve the optional semicolon problem (see Section 5.5).

Even though testing of the generation of the intermediate representations and for the GCC integration is mainly in the debugging stages, certain features have been discovered that would be helpful in the future. The GCC 3.5.0 snapshot was itself built and configured with the `--enable-checking` option set, which means that all tree types are checked at runtime (though resulting in a performance penalty (Stallman 2004*a*)). This would obviously not be suitable for a release version, but would be extremely helpful in development whilst testing the tree building functionality. The file `tree-pretty-print.c` implements several debugging functions that when given a GENERIC tree node, they print a C representation of the tree. The output is not meant to be compilable C, but it would be of great help when debugging transformations from the tree building passes. Other compiler flags that could possibly be helpful for testing are `-ffump-tree-gimple`, which dumps C-like representations of the GIMPLE form and the `-d` flag which is used for getting RTL dumps at the different stages of optimisation during compilation (Stallman 2004*b*).

In summary, the testing strategy used was based around a significant amount of testing and debugging during the development phases, coupled the use of specially constructed BCPL test code examples. These test cases have attempted to utilise a wide range of BCPL language features, with the emphasis placed on creating structures likely to cause problems. Some examples of the output from these test cases can be found in Appendix F.

Chapter 7

Conclusion

7.1 Overview

The software developed as a result of this project has provided a starting framework towards creating a GCC front end for BCPL. The scope of this project from the start was massive and this was increased by some of the design decisions made. However, these choices were made because of the suitability and elegance of the solution, rather than on time or implementation constraints. The depth of the Literature Survey in Chapter 2 gave us a firm appreciation of the domain and the current status of the GCC infrastructure. It also highlighted the important issues when developing a compiler by reusing an existing infrastructure. The range of possible implementation options enabled us to compare and contrast existing compiler architectures and whether they were suitable or appropriate for this project. A summary of some of the key problems encountered during this project are discussed below:

- The main problem discovered as we advanced in the project was the incomplete nature of the documentation and resources available for GCC 3.5.0. The initial research into the domain highlighted the fact that developing a front end for GCC would not be trivial, but surprisingly for such a well managed and documented software development, the available resources for certain parts of the system were poor. This was compounded by the fact that the incomplete parts of the documentation for the 3.5.0 release were seemingly the most critical parts, insofar for building the intermediate representations and the integration of GCC. Fairly comprehensive documentation existed for much of the GIMPLE and RTL syntax and functionality, but large sections of the important GENERIC tree building functions and how you then pass these trees to the middle and back end were missing. This meant that for most of the intermediate representation development, only two current sources of information were used for reference: the GCC Internals Manual (Stallman 2004*a*) and the Tree SSA online documentation (GCC 2003*d*). It is understandable that the fast developments over the past few months with GCC and the recent mainline merge would mean that documentation would be missing, but it seems unusual that such an important change to the GCC infrastructure is not fully documented. The GCC Mission Statement (GCC 1999) declares its first design and development goal to be "new languages", but surprisingly little up-to-date documentation exists on developing new language front ends.

- It could be said that the choice of using GCC as our implementation framework was flawed, but it was felt that compared to the other potential infrastructures discussed, GCC provided the best solution. The domain research performed in Chapter 2 demonstrated that while the other options such as LLVM, lcc and SUIF were extremely well designed compiler projects, none were comparable to GCC for popularity and support. It would have been interesting to see how the other projects would have developed, but it seems that irrespective of the infrastructure used, certain BCPL language features would have been troublesome to implement. Nevertheless, the learning curve of the GCC internals was steep and attempting to understand partially implemented or poorly documented code has definitely contributed to less development than anticipated. However, this is tempered by the fact that the research performed lays the groundwork for a future implementation (Crick 2004).

- A range of problems were associated with certain BCPL language features, such as the implementation of `VALOF`, `LONGJUMP`, `LEVEL`, the global vector and the system library. It seemed that these had no trivial solution and perhaps were made harder by the modern intermediate representa-

tion chosen. Nevertheless, solutions were discussed and it seems that there may exist GENERIC nodes that could implement some of the features mentioned.

- Definite problems were encountered when attempting to formulate software engineering principles and methods for this compiler project. It seems exceptionally hard to define a reasonable set of requirements and perform elicitation and analysis. It also occasionally seemed a futile exercise attempting to provide a model to adhere to, when in most iterations, the development and design was based around past experience and knowledge of the domain and GCC. Nevertheless, a range of software engineering principles have been applied in the requirements, design and implementation, but it seemed as if the formulation of a rigorous test plan was both unnecessary and impractical.

- Due to the fact that it was necessary to stay current with the versions and snapshots released by GCC, the time taken to configure, build and install GCC became a problem. Each build took upwards of two hours, because it was necessary to build and test the entire suite. This impacted on development work, as GCC was built approximately fifteen times during the project.

- Even though it was discussed during the requirements and design phase, it was hard to ensure that a secondary plan was in place if any problems occurred. As soon as we were committed to GCC, the development had to make changes and alterations to accommodate the infrastructure, so this forced the code to be forked into two branches.

7.2 Milestones

A summary of the key milestones of this project are listed below:

- Development of an efficient Flex scanner for BCPL with support functions and lexical error handling.

- Development of a Bison-generated parser with syntactic error handling.

- Creation of a separately-chained hash table as the symbol table structure.

- Formalisation of a BNF grammar for BCPL, developed from multiple sources and incorporating some of the commonly-used extensions to the language.

- Rudimentary implementation of the BCPL system library and the global vector, enabling input/output functionality.

- Preliminary GENERIC nodes built for a subset of the BCPL language, adapted from the GCC C and Fortran front ends.

- Research into the GENERIC/GIMPLE infrastructure and the new optimisation framework for GCC, giving an understanding of the future development of GCC and the integration of new languages.

- Interaction with the GCC developer community via the GCC mailing lists, and communications with the mainline branch maintainers. There seems to have been some positive interest from the GCC community in relation to this project.

- The creation of a SourceForge community project to continue with this project in the future (Crick 2004). It should also be possible to involve other developers with the project and at some point in the future produce official releases.

- Adherence to the appropriate GNU, GCC and ANSI C coding standards.

- Successfully meeting most of the requirements set for this project. Even though this project has not been a total success, we have been able to produce some interesting information about the GCC infrastructure and have shown that this project is viable and achievable within a larger time frame.

7.3 Further Work

The system produced from the work of this dissertation fulfils a number of the objectives as specified in Chapter 1. Due to the time frame and open-ended nature of this project, there exists a lot of scope for improvements and future work. One major reimplementation would be to change the Bison-generated parser to a recursive descent parser. After the lessons learnt from developing the Bison parser, the benefits from using recursive descent would be significant and would remove a whole host of conflicts within the present parser. An important example would be to provide an elegant solution to the optional semicolon problem (see Section 5.5). Improvements could also be made to the symbol table and also how it integrates and interacts with the rest of the GCC infrastructure. It would be important to fully implement the features discussed in Section 5.6, especially improvements to the implementation of the global vector and the BCPL system library. It would also be feasible to investigate BCPL's application binary interface and attempt to create a framework where it would be possible to link in the original BCPL library at compile time and map calls to library functions as normal. Once the implementation has matured, it would also be important to develop an improved testing strategy, possibly based upon black box testing or unit testing.

Depending on the changes to GCC in the upcoming year, the `tree-ssa` branch will be released as GCC 3.5.0 (GCC 2004*b*) in early to mid 2005. Full integration for the BCPL front end would still require a large amount of development effort and also some new approaches for implementing certain language features. However, as the active development for GCC mainline advances, the improvement in the documentation may provide a better base for this to occur. To enable the continuation of this project and to take advantage of more online collaboration, a SourceForge.net[1] project has been proposed and accepted by the SourceForge maintainers and been named "BCPL for GCC" (Crick 2004). The project homepage can be accessed at `http://sourceforge.net/projects/gccbcpl/`. There has been some interest in the SourceForge project in both academic and industrial circles, so future collaboration could provide more developers working on the project. A similar project to develop a PL/1 front end for GCC (Sorensen 2004) has just released version 0.0.6 and is currently under active development. The benefits of using the SourceForge infrastructure would include the availability of source management tools such as CVS, support and release management, mailing lists, discussion forums, shell services and compile farms. The impact of these tools on the project would have undoubtedly been positive, as the use of tools like CVS would have been enormously beneficial especially when the code was forked to handle the GCC development.

7.4 Concluding Remarks

The rapid development and popularity of the system programming language C in the 1980s led to the overall demise of BCPL. However, the popularity of BCPL as an efficient and flexible programming language means that interest and affection for the language survives today. It is quite possible that if the popularity of byte-addressed minicomputers had not happened in the 1970s, UNIX would have been initially written in BCPL (Ritchie 1993). It is by no means a dead language, as large amounts of legacy code exists today and it remains a popular language in academia for research. There have been many recent developments that have involved BCPL, including an attempt to port BCPL Cintcode programs over to the Microsoft .NET Common Language Runtime (Singer 2002).

This dissertation has discussed the history and heritage of BCPL and has looked at a range of modern compiler infrastructures. The use of a modern compiler infrastructure will provide a proven framework to create a robust compilation solution for legacy BCPL users. A discussion of the methods used for the design and implementation of such a system has followed. The final system, even if unfinished, has achieved many of the objectives set out in Chapter 1 and this was briefly demonstrated in the test code examples. The use of the GCC infrastructure via the SourceForge project will provide an elegant solution to the BCPL legacy problem, and would be excellently supported by active development and modern optimisation strategies.

[1] SourceForge.net is the world's largest open source software development web site, providing free hosting to tens of thousands of projects. See `http://sourceforge.net/` for further details.

Bibliography

Aho, A. V., Sethi, R. & Ullmann, J. D. (1986), *Compilers: Principles, Techniques and Tools*, World Student Series, 1st edn, Addison-Wesley.

Aigner, G., Diwan, A., Heine, D. L., Lam, M. S., Moore, D. L., Murphy, B. R. & Sapuntakis, C. (2003), An Overview of the SUIF2 Compiler Infrastructure, Technical report, Stanford University.

Alpern, B., Wegman, M. N. & Zadeck, F. K. (1988), Detecting Equality of Variables in Programs, *in* 'Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages', ACM Press, pp. 1–11.

Barron, D. W., Buxton, J. N., Hartley, D. F., Nixon, E. & Strachey, C. (1963), 'The Main Features of CPL', *The Computer Journal* **6**(2), 134–143.

Bennett, J. P. (1996), *Introduction to Compiling Techniques: A First Course Using ANSI C, LEX and YACC*, The McGraw-Hill International Series in Software Engineering, 2nd edn, McGraw-Hill.

Chomsky, N. (1956), 'Three Models for the Description of Language', *IRE Transactions on Information Theory* **2**, 113–124.

Chomsky, N. (1959), 'On Certain Formal Properties of Grammars', *Information and Control* **2**, 137–167.

Crick, T. (2004), 'GCC for BCPL', `http://gccbcpl.sourceforge.net/` (10 May 2004).

Curry, J. F. (1969), *The BCPL Reference Manual*, Xerox PARC.

Cytron, R., Ferrante, J., Rosenn, B. K., Wegman, M. N. & Zadeck, F. K. (1991), 'Efficiently Computing Static Single Assignment Form and the Control Dependence Graph', *ACM Transactions on Programming Languages and Systems (TOPLAS)* **13**(4), 451–490.

Donnelly, C. & Stallman, R. (2002), *Bison: The YACC-compatible Parser Generator*, GNU/Free Software Foundation Inc.

Emery, G. (1986), *BCPL and C*, 1st edn, Blackwell Scientific Publications.

Feather, C. (2002), 'A Brief Description of BCPL', `http://www.lysator.liu.se/c/clive-on-bcpl.html` (10 November 2003).

Fischer, C. N. & LeBlanc, R. J. (1991), *Crafting a Compiler with C*, Programming/Compiler Design, 1st edn, Benjamin/Cummings Publishing.

FOLDOC (2004), 'Free On-Line Dictionary of Computing', `http://foldoc.doc.ic.ac.uk/foldoc/index.html` (10 November 2003).

Fraser, C. & Hanson, D. (2003), 'lcc, A Re-targetable Compiler for ANSI C', `http://www.cs.princeton.edu/software/lcc/` (20 December 2003).

Fraser, C. W. & Hanson, D. R. (1991*a*), 'A Code Generation Interface for ANSI C', *Software - Practice and Experience* **21**(9), 963–998.

Fraser, C. W. & Hanson, D. R. (1991*b*), 'A Re-targetable Compiler for ANSI C', *ACM SIGPLAN Notices* **26**(10), 29–43.

GCC (1999), 'GCC Development Mission Statement', `http://gcc.gnu.org/gccmission.html` (10 December 2003).

GCC (2003*a*), 'GCC Coding Standards', `http://gcc.gnu.org/codingconventions.html` (02 December 2003).

GCC (2003*b*), 'GCC Front Ends', `http://gcc.gnu.org/frontends.html` (20 October 2003).

GCC (2003*c*), 'SSA for Trees', `http://gcc.gnu.org/projects/tree-ssa` (20 October 2003).

GCC (2003*d*), 'Tree SSA Documentation', `http://people.redhat.com/dnovillo/tree-ssa-doc/html/index.html` (01 November 2003).

GCC (2004*a*), 'Contributing to GCC', `http://gcc.gnu.org/contribute.html` (14 January 2004).

GCC (2004*b*), 'GCC 3.5 Release Series - Changes, New Features and Fixes', `http://gcc.gnu.org/gcc-3.5/changes.html` (22 April 2004).

GCC (2004*c*), 'GCC Home Page - GNU Project - Free Software Foundation (FSF)', `http://gcc.gnu.org` (20 October 2003).

GCC (2004*d*), 'Optimize Options - Using the GNU Compiler Collection (GCC)', `http://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html` (01 May 2004).

GNU (2003*a*), 'GNU Coding Standards', `http://www.gnu.org/prep/standards_toc.html` (02 December 2003).

GNU (2003*b*), 'GNU General Public License', `http://www.gnu.org/copyleft/gpl.html` (10 December 2003).

GNU (2003*c*), 'GNU Pascal', `http://www.gnu-pascal.de/gpc/index-orig.html` (02 April 2004).

GNU (2003*d*), 'Information For Maintainers of GNU Software', `http://www.gnu.org/prep/maintain.html` (10 December 2003).

GNU (2004*a*), 'GNU Fortran 95', `http://gcc.gnu.org/fortran/` (02 April 2004).

GNU (2004*b*), 'GNU Modula-2', `http://floppsie.comp.glam.ac.uk/Glamorgan/gaius/web/GNUModula2.html` (02 April 2004).

Grune, D., Bal, H. E., Jacobs, C. J. & Langendoen, K. G. (2000), *Modern Compiler Design*, Worldwide Series in Computer Science, 1st edn, John Wiley & Sons Ltd.

Hendren, L., Donawa, C., Emami, M., Gao, G. & Sridharan, B. (1992), Designing the McCAT Compiler Based on a Family of Structured Intermediate Representations, *in* 'Proceedings of the 5th International Workshop on Languages and Compilers for Parallel Computing', number 457 *in* 'Lecture Notes in Computer Science', Springer-Verlag, pp. 406–420.

IEEE (1998), *IEEE/ANSI 830-1998 Recommended Practice for Software Requirements Specifications*, The Institute of Electrical and Electronics Engineers.

IOCCC (2003), 'International Obfuscated C Code Contest', `http://www.ioccc.org/main.html` (10 December 2003).

ISO (1999), *ISO/IEC 9899:1999: Programming Languages – C*, International Organisation for Standardisation.

ISO (2003), *ISO/IEC 9945:2003: Portable Operating System Interface (POSIX)*, International Organisation for Standardisation.

Johnson, S. C. (1975), Yacc - Yet Another Compiler Compiler, Technical report, Bell Laboratories.

Johnson, S. C. & Kernighan, B. W. (1973), *The Programming Language B*, Bell Laboratories.

Johnson, S. C. & Ritchie, D. M. (1978), 'UNIX Time-Sharing System: Portability of C Programs and the UNIX System', *AT&T Bell Laboratories Technical Journal* **57**(6), 2021–2048.

Josling, T. (2001), 'Using, Maintaining and Enhancing COBOL for the GNU Compiler Collection (GCC)', `http://cobolforgcc.sourceforge.net/cobol_toc.html` (02 April 2004).

Kernighan, B. W. & Ritchie, D. M. (1978), *The C Programming Language*, 1st edn, Prentice Hall.

Lattner, C. (2004), 'The LLVM Compiler Infrastructure', `http://llvm.cs.uiuc.edu/` (01 February 2004).

Lattner, C. & Adve, V. (2004*a*), LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation, *in* 'Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO'04)', Palo Alto, California.

Lattner, C. & Adve, V. (2004*b*), 'LLVM Language Reference Manual', `http://llvm.cs.uiuc.edu/docs/LangRef.html` (01 February 2004).

Lattner, C., Dhurjati, D. & Stanley, J. (2004), 'LLVM Programmer's Manual', `http://llvm.cs.uiuc.edu/docs/ProgrammersManual.html` (01 February 2004).

Lesk, M. E. & Schmidt, E. (1975), Lex - A Lexical Analyzer Generator, Technical report, Bell Laboratories.

Levine, J. R., Mason, T. & Brown, D. (1995), *lex & yacc*, Unix Programming Tools, 2nd edn, O'Reilly.

Louden, K. C. (1997), *Compiler Construction: Principles and Practice*, 1st edn, PWS Publishing.

Loudon, K. (1999), *Mastering Algorithms with C*, Programming Series, 1st edn, O'Reilly.

Manning, M. (1999), 'BCPL', `http://www.cus.cam.ac.uk/~mrm1/` (10 November 2003).

Merill, J. (2002*a*), 'Language-independent functions-as-trees representation', `http://gcc.gnu.org/ml/2002-07/msg00890.html` (20 November 2003).

Merill, J. (2002*b*), 'Re: Language-independent functions-as-trees representation', `http://gcc.gnu.org/ml/gcc/2002-08/msg01397.html` (20 November 2003).

Merrill, J. (2003), GENERIC and GIMPLE: A New Tree Representation for Entire Functions, *in* 'Proceedings of the 2003 GCC Summit', pp. 171–180.

Middleton, M. D., Richards, M., Firth, R. & Willers, I. (1982), A Proposed Definition of the Language BCPL, Technical report.

Mycroft, A. & Norman, A. C. (1986), *The Internal Structure of the Norcroft C Compiler*, Codemist Ltd.

Naur, P., Backus, J. W. & McCarthy, J. (1963), 'Revised Report on the Algorithmic Language ALGOL 60', *The Computer Journal* **5**(4), 349–357.

Novillo, D. (2003*a*), Tree SSA - A New Optimization Infrastructure for GCC, *in* 'Proceedings of the 2003 GCC Summit', pp. 181–194.

Novillo, D. (2003*b*), Tree SSA: A New High-Level Optimization Framework for the GNU Compiler Collection, Technical report, Red Hat Canada Ltd.

Page, D. (2003), BCPL Compiler. BSc (Hons) Computer Science dissertation, University of Bath.

Paxson, V. (1995), *Flex: A Fast Scanner Generator*, GNU/Free Software Foundation Inc.

Pizka, M. (1997), Design and Implementation of the GNU INSEL Compiler, Technical Report TUM-I9713, Technische Universität München.

Pressman, R. (2000), *Software Engineering: A Practitioner's Approach*, European Adaptation, 5th edn, McGraw-Hill Education.

Richards, M. (1967), *The BCPL Reference Manual*, Massachusetts Institute of Technology.

Richards, M. (1969), BCPL: A Tool for Compiler Writing and System Programming, *in* 'Proceedings of the AFIPS Spring Joint Computer Conference', Vol. 34, American Federation of Information Processing Societies, pp. 557–566.

Richards, M. (1971), 'The Portability of the BCPL Compiler', *Software - Practice and Experience* **1**(2), 135–146.

Richards, M. (1973), *The BCPL Programming Manual*, University of Cambridge.

Richards, M. (1975), INTCODE - An Interpretive Machine Code for BCPL, Technical report, University of Cambridge.

Richards, M. (2000), 'BCPL', `http://www.cl.cam.ac.uk/users/mr/BCPL.html` (20 October 2003).

Richards, M. (2003), *The BCPL Cintcode System Users Guide*, University of Cambridge.

Richards, M. & Whitby-Strevens, C. (1980), *BCPL - the language and its compiler*, 1st edn, Cambridge University Press.

Ritchie, D. M. (1993), 'The Development of the C Language', *ACM SIGPLAN Notices* **28**(3), 201–208. Preprints of the ACM History of Programming Languages Conference HOPLII.

Ritchie, D. M. & Thompson, K. (1974), 'The UNIX Time-Sharing System', *Communications of the ACM* **17**(7), 365–375.

Rosen, B. K., Wegman, M. N. & Zadeck, F. K. (1988), Global Value Numbers and Redundant Computations, *in* 'Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages', ACM Press, pp. 12–27.

Singer, J. (2002), Porting Legacy Interpretive Bytecode to the CLR, *in* 'Proceedings of the Inaugural Conference on the Principles and Practice of Programming', ACM International Conference Proceeding Series, National University of Ireland Publishing, pp. 163–168.

Sommerville, I. (2001), *Software Engineering*, International Computer Science Series, 6th edn, Addison-Wesley.

Sorensen, H. (2004), 'PL/1 for GCC', `http://pl1gcc.sourceforge.net/` (02 March 2004).

Stallman, R. M. (2004*a*), *GNU Compiler Collection Internals*, GNU/Free Software Foundation Inc.

Stallman, R. M. (2004*b*), *Using the GNU Compiler Collection*, GNU/Free Software Foundation Inc.

SUI (2001), 'The SUIF 2 Compiler System', `http://suif.stanford.edu/suif/suif2/` (21 December 2003).

Tremblay, J.-P. & Sorenson, P. G. (1985), *The Theory and Practice of Compiler Writing*, Computer Science Series, 1st edn, McGraw-Hill.

Wikipedia (2004), 'Wikipedia, the free encyclopedia', `http://en.wikipedia.org/` (10 November 2003).

Wilhelm, R. & Maurer, D. (1995), *Compiler Design*, International Computer Science Series, 1st edn, Addison-Wesley.

Willers, I. & Mellor, S. (1976), *The BCPL Mini-Manual*, Data Handling Division, CERN.

Appendix A

Compiler Phases

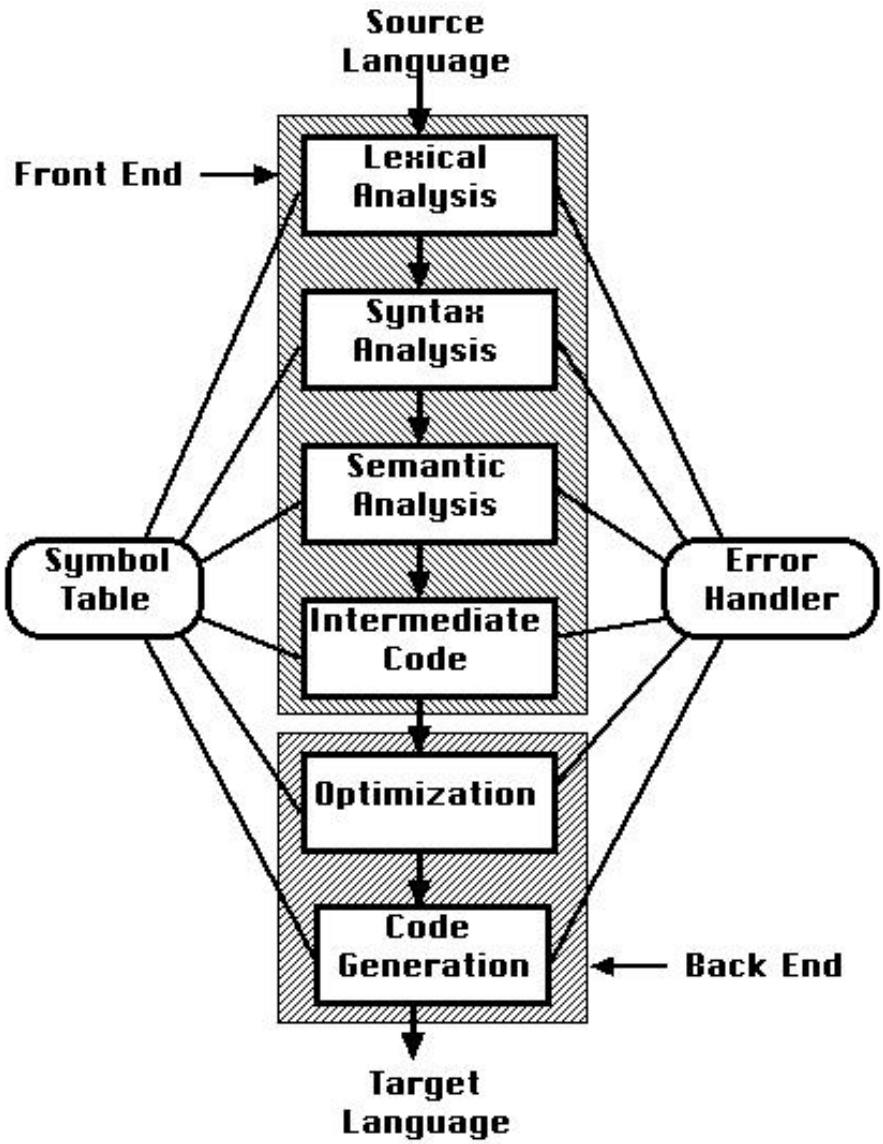


Figure A.1: Basic compiler phases

Appendix B

BCPL, B and C Code Examples

```
1   GET "LIBHDR"
2
3   LET START () BE
4   $(
5       WRITES ("Hello, world!*N")
6   $)
```

Figure B.1: BCPL code example

```
1   main()
2   {
3       putstr("Hello, world!"); putchar('*n');
4   }
```

Figure B.2: B code example

```
1   #include <stdio.h>
2
3   int main(void)
4   {
5       printf("Hello, world!\n");
6       return 0;
7   }
```

Figure B.3: C code example

Appendix C

Backus-Naur Form Grammar for BCPL

This section presents the Backus-Naur Form (BNF) grammar for BCPL. As mentioned previously, this version of the BCPL BNF grammar has been taken from (Richards & Whitby-Strevens 1980), but does not reflect the changes with respect to the commonly-used extensions, such as the optional semicolons or the infix byte operator. At the outermost level, a BCPL program is a sequence of declarations (Richards & Whitby-Strevens 1980).

Identifiers, strings, numbers

```
<letter>        ::= A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z
<octal digit>   ::= 0|1|2|3|4|5|6|7
<hex digit>     ::= 0|1|2|3|4|5|6|7|8|9|A|B|C|D|E|F
<digit>         ::= 0|1|2|3|4|5|6|7|8|9
<string const>  ::= "<255 or fewer characters>"
<char const>    ::= '<one character>'
<octal number>  ::= #<octal digit>[<octal digit>]
<hex number>    ::= #X<hex digit>[<hex digit>]
<number>        ::= <octal number> | <hex number> | <digit>[<digit>]
<identifier>    ::= <letter>[<letter> | <digit>| .]
```

Operators

```
<address op>    ::= @ | !
<mult op>       ::= * | / | REM
<add op>        ::= + | -
<rel op>        ::= = | = | <= | >= | < | >
<shift op>      ::= << | >>
<and op>        ::= &
<or op>         ::= |
<eqv op>        ::= EQV | NEQV
<not op>        ::=
```

Expressions

```
<element>       ::= <char const> | <string const> | <number> | <identifier> | TRUE | FALSE
<primary E>     ::= <primary E>(<expr list>) | <primary E>( ) | (<expression>) | <element>
<vector E>      ::= <vector E> ! <primary E> | <primary E>
<address E>     ::= <address op><address E> | <vector E>
<mult E>        ::= <mult E><mult op><address E> | <address E>
<add E>         ::= <add E><add op><mult E> | <add op><mult E> | <mult E>
<rel E>         ::= <add E>[<rel op><add E>]
<shift E>       ::= <shift E><shift op><add E> | <rel E>
<not E>         ::= <not op><shift E> | <shift E>
<and E>         ::= <not E>[<and op><not E>]
<or E>          ::= <and E>[<or op><and E>]
```

```
<eqv E>          ::= <or E>[<eqv op><or E>]
<conditional E>  ::= <eqv E> -> <conditional E>,<conditional E> | <eqv E>
<expression>     ::= <conditional E> | TABLE <const expr>[,<const expr>] |
                     VALOF <command>
```

Constant expressions

```
<c element>      ::= <char const> | <number> | <identifier> | TRUE | FALSE |
                     (<constant expressions>)
<c mult E>       ::= <c mult E><mult op><c element> | <c element>
<c add E>        ::= <c add E><add op><c mult E> | <add op><c mult E> | <c mult E>
<c shift E>      ::= <c shift E><shift op><c add E> | <c add E>
<c and E>        ::= <c and E><and op><c shift E> | <c shift E>
<const expr>     ::= <const expr><or op><c and E> | <c and E>
```

Lists of expressions and identifiers

```
<expr list>      ::= <expression>[,<expression>]
<name list>      ::= <name>[,<name>]
```

Declarations

```
<manifest item> ::= <identifier>=<const expr>
<manifest list> ::= <manifest item>[;<manifest item>]
<manifest decl> ::= MANIFEST$(<manifest list>$)
<static decl>   ::= STATIC$(<manifest list>$)
<global item>   ::= <identifier>:<const expr>
<global list>   ::= <global item>[;<global item>]
<global decl>   ::= GLOBAL$(<global list>$)
<simple defn>   ::= <name list>=<expr list>
<vector defn>   ::= <identifier>=VEC <const expr>
<function defn> ::= <identifier>(<name list>)=<expression> |
                    <identifier>( )=<expression>
<routine defn>  ::= <identifier>(<name list>) BE <command> |
                    <identifier> ( ) BE <command>
<definition>    ::= <simple defn> | <vector defn> | <function defn> | <routine defn>
<simult decl>   ::= LET <definition>[ AND <definition>]
<declaration>   ::= <simult decl> | <manifest decl> | <static decl> | <global decl>
```

Left hand side expressions

```
<lhse>           ::= <identifier> | <vector E> ! <primary E> | !<primary E>
<lhs list>       ::= <lhse>[,<lhse>]
```

Unlabelled commands

```
<assignment>     ::= <left hand side list>:=<expr list>
<simple cmd>     ::= BREAK | LOOP | ENDCASE | RETURN | FINISH
<goto cmd>       ::= GOTO <expression>
<routine cmd>    ::= <primary E>(<expr list>) | <primary E> ( )
<resultis cmd>   ::= RESULTIS <expression>
<switchon cmd>   ::= SWITCHON <expression> INTO <command command>
<repeatable cmd>::= <repeatable cmd> REPEAT | <repeatable cmd> REPEATUNTIL <expression> |
                    <repeatable cmd> REPEATWHILE <expression>
<until cmd>      ::= UNTIL <expression> DO <command>
<while cmd>      ::= WHILE <expression> DO <expression>
<for cmd>        ::= FOR <identifier> = <expression> TO <expression>
                         BY <const expr> DO <command> |
                    FOR <identifier> = <expression> TO <expression> DO <command>
<repetitive cmd>::= <repeated command> | <until cmd> | <while cmd> | <for cmd>
<test cmd>       ::= TEST <expression> THEN <command> ELSE <command>
<if cmd>         ::= IF <expression> THEN <command>
<unless cmd>     ::= UNLESS <expression> THEN <command>
<unlabelled cmd>::= <repeatable cmd> | <repetitive cmd> | <test cmd> | <if cmd>
```

Labelled commands

```
<label prefix>  ::= <identifier> :
<case prefix>   ::= CASE <const expr> :
<default prefix>::= DEFAULT :
<prefix>        ::= <label prefix> | <case prefix> | <default prefix>
<command>       ::= <unlabelled cmd> | <prefix> <command> | <prefix>
```

Blocks and compound statements

```
<command list>  ::= <command>[;<command>]
<decl part>     ::= <declaration>[;<declaration>]
<block>         ::= $( <decl part> ; <command list> $)
<compound cmd>  ::= $( <command list> $)
<program>       ::= <decl part>
```

Appendix D

BCPL Operator Precedence

BCPL operators in order of precedence (highest to lowest, operators near the top bind most tightly):

Operators	Synonyms	Associativity
function call		left
`!` (dyadic) `% ::`	`OF`	left
`@ !` (monadic)		left
`* / REM`		left
`+ -` (dyadic and monadic)		left
`= ~= > < >= <=`	`EQ NE = GT LT GE LE`	left
`<< >>`	`LSHIFT RSHIFT`	left
`NOT`	`~`	left
`&`	`/\ LOGAND`	left
`|`	`\/ LOGOR`	left
`EQV NEQV`		left
→		right
`TABLE`		right
`VALOF`		left
`SLCT`		left

Figure D.1: BCPL operator precedence, associativity and common synonyms

The floating point operators (beginning with #) have the same precedence as their integer equivalents, while the precedence for the field selector (`SLCT`) is not described in (Richards & Whitby-Strevens 1980). All operators associate left-to-right except for → and `TABLE`.

Appendix E

Integration of GENERIC and GIMPLE into GCC

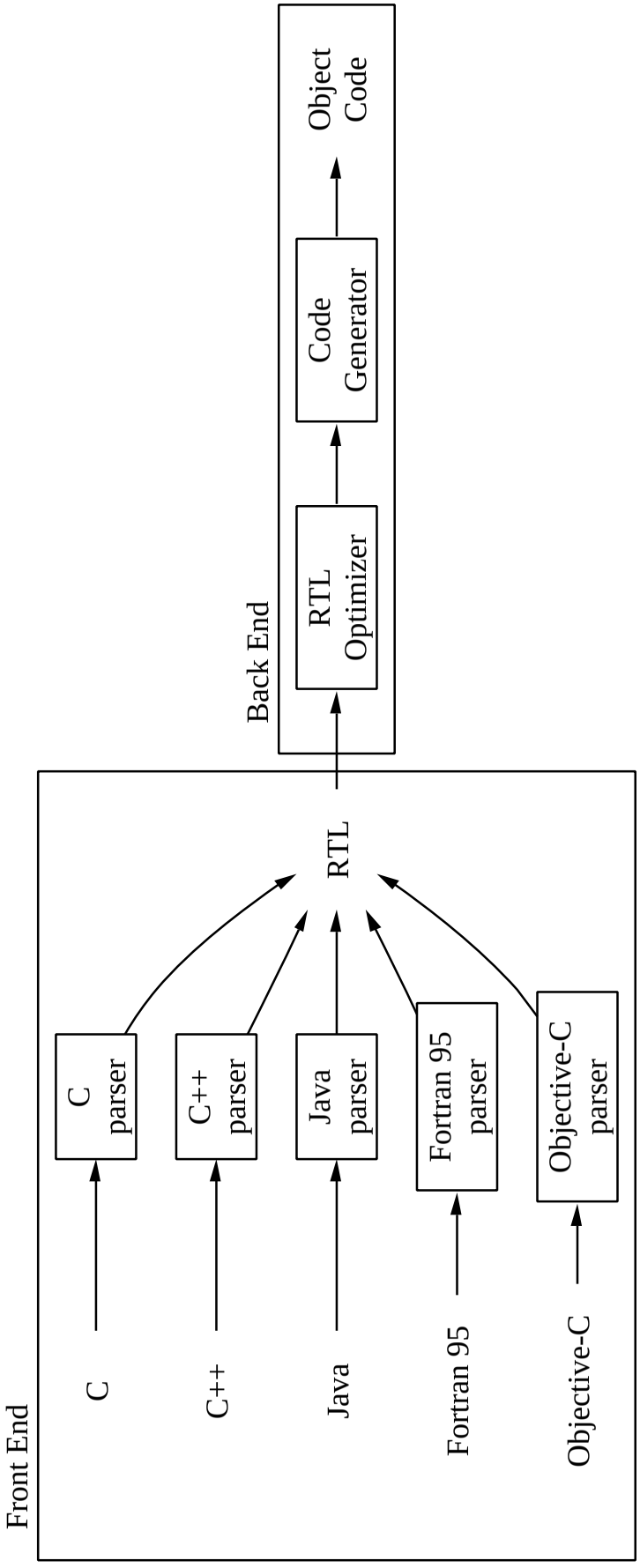


Figure E.1: Existing GCC framework (Novillo 2003*a*)

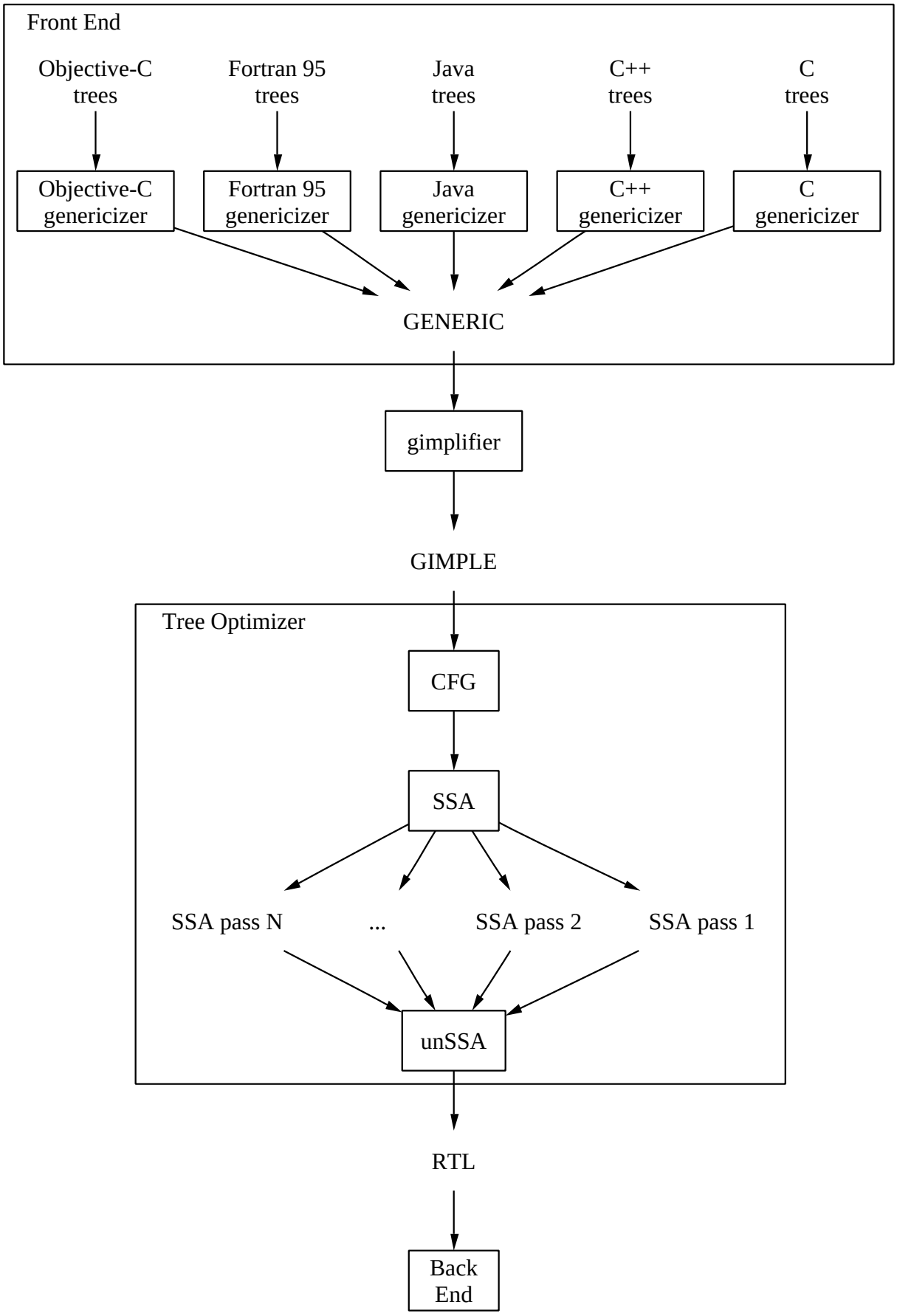


Figure E.2: Proposed integration of GENERIC and GIMPLE within the GCC framework (Novillo 2003*a*)

Appendix F

Test Results

This chapter displays a short selection of some of the test scripts used and results from running the compiler. For further details, please see the attached source code CD in Appendix H or see the SourceForge project home page (Crick 2004).

```
[cs1tc@tcrick bcpl]$ ./bcpl
Usage: bcpl [OPTION...] file
Try ‘bcpl --help’ or ‘bcpl --usage’ for more information.
...
...
[cs1tc@tcrick bcpl]$ ./bcpl --help
Usage: bcpl [OPTION...] file
A BCPL compiler

  -o, --output=<file>        Place the output into <file>
  -v, -s, --verbose, --silent   Produce verbose output
  -?, --help                 Give this help list
      --usage                Give a short usage message
  -V, --version              Print program version

Mandatory or optional arguments to long options are also
mandatory or optional for any corresponding short options.
Report bugs to <tc@cs.bath.ac.uk>.
...
...
[cs1tc@tcrick bcpl]$ ./bcpl --usage
Usage: bcpl [-vs?V] [-o <file>] [--output=<file>] [--verbose]
            [--silent] [--help] [--usage] [--version] file
...
...
[cs1tc@tcrick bcpl]$ ./bcpl --version
bcpl (BCPL) version 0.0.1 20040513
Copyright (C) 2004 Tom Crick (tc@cs.bath.ac.uk)
This is free software; see the source for copying conditions.
There is NO warranty; not even for MERCHANTABILITY or FITNESS
FOR A PARTICULAR PURPOSE.
...
...
[cs1tc@tcrick pre_trees]$ ./bcpl ../bcpltest/ackermann.b
../bcpltest/ackermann.b:144:20 parse error,
                        unexpected PERCENT
../bcpltest/ackermann.b:144:50 multi-line
                        string constants are not allowed

// ackermann.b
GET "LIBHDR"
```

```
LET A(M,N) = M=0 -> N+1, N=0 -> A(M-1,1), A(M-1, A(M,N-1))
AND START() BE
$(
        LET M, N = 0, 0
        $(
                WRITES("Type two arguments (zero to exit)*N")
                M := READN()
                N := READN()
                WRITEF(Ackermann(%N, %N) = %N*N", M, N, A(M,N))
        $)REPEATUNTIL M = 0
$)
...
...
[cs1tc@tcrick pre_trees]$ ./bcpl ../bcpltest/hanoi.b
../bcpltest/hanoi.b:26:7 parse error,
                        unexpected IDENTIFIER, expecting PLING

// hanoi.b
GLOBAL $(
  START:1;
  WRITEF:76
$)

LET MOVEIT(F, T) BE $(
  WRITEF("move %N --> %N*N", F, T)
$)

LET HANOI(N, T, F, U) BE $(
  IF N=0 RETURN;
  HANOI(N-1, U, F, T);
  MOVEIT(F, T);
  HANOI(N-1, T, U, F)
$)

LET START () BE $(1
  HANOI(1, 3, 1, 2)
FINISH $)1
...
...
[cs1tc@tcrick pre_trees]$ ./bcpl ../bcpltest/fastackermann.b

// fastackermann.b
GET "LIBHDR"

MANIFEST $( maxn = 100 $)

LET A(T,M,N) = VALOF
$(
        LET x = M < 4 & N < maxn -> T!(M*maxn + N), 0
        IF x = 0 THEN
        $(
                TEST M = 0 THEN x := N+1
                ELSE x := N = 0 -> A(T, M-1, 1),
                                   A(T, M-1, A(T, M, N-1))
                IF M<4 & N<maxn THEN T!(M*maxn + N) := x
        $)
        RESULTIS x
$)
```

```
AND START() BE
$(
        LET V = VEC 4*maxn
        LET M, N = 0, 0
        $(
                FOR j = 0 TO 4*maxn DO V!j := 0 //clear array
                WRITES("Type two arguments (zero to exit)*N")
                M := READN()
                N := READN()
                WRITEF("Ackermann(%N, %N) = %N*N",
                                        M, N, A(V,M,N))
        $) REPEATUNTIL M=0
$)
```

Appendix G

Program Versions

The program versions of the tools used in this project are listed below.

Red Hat Linux 9 (kernel 2.4.20-31.9)
GNU Flex 2.5.4
GNU Bison 1.35
GNU Make 3.79.1
GNU gdb 5.3post-0.20021129.18rh
GNU DDD 3.3.1
GNU Emacs 21.2.1

```
GCC v3.5-tree-ssa
Reading specs from:
/usr/local/gccssa/lib/gcc/i686-pc-linux-gnu/
3.5-tree-ssa/specs
Configured with:
../gcc-tree-ssa-20040407-src/configure
--enable-shared
--enable-threads=posix
--enable-checking
--with-system-zlib
--enable-__cxa_atexit
--program-suffix=ssa
--prefix=/usr/local/gccssa
Thread model: posix
gcc version 3.5-tree-ssa 20040506 (merged 20040430)
```

For typesetting and referencing this document:
LaTeX (Web2C 7.3.1) 3.14159 (kpathsea version 3.3.1)
BibTeX (Web2C 7.3.1) 0.99c (kpathsea version 3.3.1)
Kile 1.6.1 (LaTeX front end for KDE, available from `http://kile.sourceforge.net/`)

Appendix H

Source Code

As previously discussed, the code has been forked into two development branches: one (frozen) branch containing the basic scanner, parser and symbol table; the other containing the GCC development, including the basic BCPL system library and global vector. The code given in the `gcc/gcc/` directory contains the GCC `3.5-tree-ssa 20040506 (merged 20040430)` source. See `http://gcc.gnu.org/onlinedocs/gcc-3.4.0/gccint/Subdirectories.html` for details of the structure of the source directory. To test the code and the compiler, a configured and built snapshot of the GCC mainline is required (available for download from (GCC 2004*c*)).

Directory structure:

```
.
|_____ docs/
|_____ pre_trees/
|_____ gcc/
        |_____ gcc/
        |      |_____ bcpl/
        |      |_____ ...
        |_____ ...
```